

**DEVELOPMENT OF A CLOUD-BASED IOT AND MACHINE  
LEARNING SYSTEM FOR INTELLIGENT MEDICAL TRIAGE AND  
PATIENT PROFILING**

**BY**

**ACHOEZEAKOR SOMTOCHUKWU PHILIP**

**20201248522**

**AND**

**CHINEDU ABRAHAM OBINNA**

**20201240342**

**A PROJECT PRESENTED TO THE DEPARTMENT OF SOFTWARE  
ENGINEERING, FEDERAL UNIVERSITY OF TECHNOLOGY,  
OWERRI**

**IN PARTIAL FULFILMENT FOR THE AWARD OF  
BACHELOR OF TECHNOLOGY (B-TECH) IN  
SOFTWARE ENGINEERING**

**SEPTEMBER 2025**

## CERTIFICATION

I certify that this project “Development of a Cloud-Based IoT and Machine Learning System for Intelligent Medical Triage and Patient Profiling” was carried out by Achoezeakor Somtochukwu Phillip (20201248522) and Chinedu Abraham Obinna (20201240342) in partial fulfilment for the award of the degree of B-Tech in Software Engineering, of the Federal University of Technology, Owerri.

---

Engr. Dr. A. I. Erike  
(Project Supervisor)

---

Date

---

Dr. C.O. Ikerionwu  
(HOD, SOE)

---

Date

---

Prof. U. F. Eze.  
(Dean, SICT)

---

Date

---

(External Examiner)

---

Date

## **DEDICATION**

This work is dedicated first and foremost to God Almighty, the ultimate source of wisdom, strength, and grace. It is through His divine guidance and mercy that this project has come to fruition.

To our dear parents, whose endless love, prayers, and sacrifices have played a pivotal role in shaping who we are today, we offer our heartfelt thanks. Your unwavering support and belief in us have been our foundation, and this work stands as a testament to your faith in us.

## **ACKNOWLEDGEMENT**

We are profoundly grateful to God Almighty for His abundant grace, strength, and guidance throughout this project. His presence has been the cornerstone of our journey, and we are thankful for His unwavering support.

Our sincere appreciation goes to our project supervisor, Engr. Dr. A. I. Erike, whose invaluable support, insightful feedback, and patient supervision were instrumental in the successful completion of this work. His dedication and encouragement have been key to our progress.

We also extend our heartfelt gratitude to the Head of Department, Dr. C. O. Ikerionwu, for his exceptional leadership and continuous support throughout our academic journey.

To our peers and fellow students, thank you for the shared experiences, intellectual contributions, and encouragement that made this academic pursuit more meaningful and fulfilling.

Finally, we appreciate the efforts and contributions of the FUTO administration for providing an enabling environment and resources necessary for academic growth and research development.

## **ABSTRACT**

This project presents the design and implementation of a cloud-based Internet of Things (IoT) and machine learning system for intelligent medical triage and patient profiling. The study addresses the persistent problem of delayed medical response and human error in manual triage processes, particularly within resource-limited healthcare environments. The proposed system integrates IoT wearable sensors, a Python-based API gateway, and an AI inference engine powered by a medical Large Language Model (LLM) to automate real-time patient assessment. Using the Prototyping Model and Structured Systems Analysis and Design Methodology (SSADM), patient vitals such as temperature and blood pressure were acquired through an ESP32 NodeMCU and transmitted via HTTPS to a Kafka-based cloud infrastructure for distributed analysis. The LLM interprets physiological and textual patient data using pattern recognition and heuristic reasoning to categorize patients into critical, urgent, or stable classes, while generating concise medical summaries. Experimental evaluation demonstrated an average triage latency of 2.3 seconds and high classification accuracy under concurrent multi-patient scenarios. The system proved scalable, secure, and reliable for real-time medical decision support, offering a cost-effective and intelligent alternative to traditional triage systems. These findings underscore the potential of IoT and AI integration in enhancing emergency response efficiency and healthcare accessibility in developing regions.

## TABLE OF CONTENT

|                                           |        |
|-------------------------------------------|--------|
| CHAPTER ONE (INTRODUCTION)-----           | 1      |
| 1.1 Background of Study -----             | 1      |
| 1.2 Problem Statement -----               | 2      |
| 1.3 Objectives of Study -----             | 3      |
| 1.4 Scope of Study -----                  | 3      |
| 1.5 Limitations of The Study -----        | 3      |
| 1.6 Significance of The Study -----       | 4      |
| 1.7 Definition of Terms -----             | 4      |
| <br>CHAPTER TWO (LITERATURE REVIEW) ----- | <br>6  |
| 2.1 Conceptual Framework -----            | 6      |
| 2.2 Theoretical Framework -----           | 7      |
| 2.3 Empirical Framework -----             | 9      |
| 2.4 Summary of Literature review -----    | 12     |
| <br>CHAPTER THREE -----                   | <br>14 |
| 3.0 Choice of Methodology -----           | 14     |
| 3.1 Data Collection -----                 | 14     |
| 3.2 Prototype Design -----                | 16     |
| 3.3 System Architecture -----             | 17     |

|                                         |        |
|-----------------------------------------|--------|
| 3.4 Profiling Algorithm -----           | 20     |
| <br>CHAPTER FOUR -----                  | <br>30 |
| 4.1 System Overview -----               | 31     |
| 4.2 Prototype Development -----         | 32     |
| 4.3 Test-led Arrangement -----          | 34     |
| 4.4 Priority Results -----              | 34     |
| 4.5 Advantages of the System -----      | 35     |
| 4.6 System Testing and Validation ----- | 37     |
| <br>CHAPTER FIVE -----                  | <br>43 |
| 5.1 Summary of Findings -----           | 43     |
| 5.2 Conclusions -----                   | 44     |
| 5.3 Recommendations -----               | 45     |
| 5.4 Contribution to Knowledge -----     | 46     |
| 5.5 Future of Work -----                | 48     |
| REFERENCES -----                        | 50     |
| APPENDIX -----                          | 52     |

## **LIST OF TABLES**

|                                                   |    |
|---------------------------------------------------|----|
| Table 4.1: Temperature Sensor Performance Profile | 34 |
|---------------------------------------------------|----|



## **LIST OF FIGURES**

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| Figure 2.1: Conceptual framework illustrating the evolution of medical triage | 6  |
| Figure 3.1 Agile methodology and OOAD                                         | 13 |
| Figure 4.1 System class diagram                                               | 17 |
| Figure 4.2 System Flowchart                                                   | 18 |
| Figure 4.3 System Architecture Diagram                                        | 18 |
| Figure 4.4 System Activity Diagram                                            | 19 |
| Figure 4.5 System Usecase Diagram                                             | 19 |

# CHAPTER ONE

## INTRODUCTION

### 1.1 Background of study

Healthcare is a critical sector in Africa, where a significant portion of the population faces challenges in accessing timely and effective medical care. With over 1.2 billion people across diverse countries, Africa is burdened with a high prevalence of communicable diseases, insufficient healthcare infrastructure, and a shortage of trained medical professionals, especially in rural areas (Health | African Development Bank Group, n.d.; Scarcity Breeds Innovation - The Rising Adoption of Health Tech in Africa, n.d.). These challenges have contributed to delayed medical response times, leading to worsened health outcomes and avoidable fatalities.

The concept of medical triage refers to the process of rapidly assessing patients and assigning them an acuity level to ensure that those with the most urgent medical needs receive care first (Hinson et al., 2025). This practice originated during wartime medicine in the 18th and 19th centuries, most notably by Dominique Jean Larrey, a French military surgeon during the Napoleonic Wars, who introduced systematic prioritization of the wounded based on urgency of care (Iserson & Moskop, 2007). In modern healthcare, triage has become more structured through the use of standardized scales such as the Emergency Severity Index (ESI), the Korean Triage and Acuity Scale (KTAS), and other region-specific models (Khosravi et al., 2024; Määttä et al., 2023; Porto, 2024). These frameworks guide clinicians in categorizing patients into priority levels using vital signs, symptoms, and observed severity. However, even with such guidelines, triage remains largely dependent on human judgment, which can introduce variability, especially in high-volume emergency departments. Recent studies have shown that while structured triage improves uniformity, it still suffers from limitations such as delays, misclassification, and bias across patient groups (Hinson et al., 2025). [Click or tap here to enter text.](#)

In many African nations, the medical triage process relies heavily on manual assessments by healthcare workers, often under resource constraints (Maskew et al., 2023). In emergency situations, the need for rapid decision-making is crucial, yet traditional triage methods can be

slow, error-prone, and dependent on the availability of trained professionals. Furthermore, in remote regions where healthcare facilities are scarce, the lack of real-time data and efficient patient profiling further exacerbates these issues, leaving patients at risk of receiving inadequate or delayed care.

Recent advancements in Internet of Things (IoT) and machine learning technologies have demonstrated potential in addressing these challenges by facilitating real-time health monitoring, data collection, and intelligent decision-making. IoT devices can now capture critical patient information, such as vital signs and environmental factors, and transmit this data to the cloud for analysis. Machine learning algorithms can then process this data to prioritize patients based on the severity of their conditions, providing healthcare workers with actionable insights for better triage and resource allocation.

Hence, this study aims to develop and implement a cloud-based IoT and machine learning system for intelligent medical triage and patient profiling. The system will leverage IoT devices to monitor real-time patient data, which will then be analysed using machine learning models to prioritize patients according to their health conditions. This system could significantly enhance the efficiency and effectiveness of healthcare delivery in African regions with limited resources, by enabling a faster and a more accurate triage process.

## **1.2 Problem statement**

In many African regions particularly in rural and underserved communities the inefficiency of medical triage systems has had a profound impact on patient outcomes. With limited resources and a shortage of trained healthcare professionals, the process of assessing and prioritizing patients often depends on manual methods that are prone to delays and errors. As a result, critically ill patients may not receive timely care, while others may be misdiagnosed or improperly prioritized. This has led to preventable complications, increased mortality rates, and overwhelming pressure on already fragile healthcare facilities. The lack of intelligent, data-driven triage solutions continues to compromise the quality and equity of healthcare delivery across the continent.

### **1.3 Objective of study**

The primary objective of this study is to develop a cloud-based IoT and machine learning system for intelligent medical triage and patient profiling.

The specific objectives of this study are to:

1. Perform requirements engineering in order to understand the user and system requirements.
2. Design the system architecture depicting different modules and their interactions.
3. Implement the autoencoder model which powers the system using machine learning and deep learning technologies such as: python, tensorflow
4. Assemble the wearable device using a NODEMCU, temperature and pressure sensors.
5. Test the performance of the system using simulated temperature and pressure sensor data as a proof of concept.

### **1.4 Scope of the study**

This study is centred on the design and implementation of a cloud-based system that combines Internet of Things (IoT) technology and machine learning to enhance medical triage and patient profiling. This will develop an IoT-based prototype module that monitors the sensor and pressures of the patient and stores the same in the cloud environment using HTTPS request. The analysis of the data will be performed in the cloud, and then reported back to the user interface for analysis and use. Only two parameters of interest – temperature and pressure will be taken into consideration.

### **1.5 Limitations of the study**

Despite its potential, this study has certain limitations. The model will be limited to patient data obtained from just a temperature and a pressure sensor, the pressure sensor used is a piezoresistive sensor which presents certain measurement inefficiencies. Hence, the aim is to prove the point that patient data can be collected remotely and used to profile patients for treatment in over-sourced facilities.

## 1.6 Significance of the Study

This study is significant to healthcare workers, particularly those in resource-constrained environments, as it provides an automated, data-driven triage system that supports faster, more accurate patient assessments and reduces the cognitive burden during emergencies. It directly benefits patients by improving access to timely and prioritized medical care, thereby increasing the chances of better health outcomes, especially in underserved areas. Health administrators stand to gain from more efficient patient flow management and smarter allocation of limited medical resources. Lastly, researchers and developers in the fields of artificial intelligence, Internet of Things (IoT), and digital health will find this study valuable as it contributes to innovation in intelligent healthcare systems tailored for developing regions.

## 1.8 Definition of terms

1. *Internet of Things (IoT)*: A network of interconnected physical devices embedded with sensors, software, and other technologies that enable them to collect, transmit, and exchange data over the internet.
2. *Medical Triage*: A process used in healthcare to assess and categorize patients based on the urgency of their medical conditions, allowing medical staff to prioritize treatment for those in most critical need.
3. *Patient Profiling*: The systematic collection and analysis of a patient's medical data—such as vital signs, symptoms, and history—to create a comprehensive overview that supports diagnosis, triage, and personalized care decisions.
4. *Machine Learning*: A branch of artificial intelligence that enables computer systems to learn patterns from data and make predictions or decisions without being explicitly programmed for every task.
5. *Autoencoder Model*: A type of neural network used in machine learning for unsupervised learning. It is designed to learn efficient data encodings by compressing input data into a lower-dimensional representation and then reconstructing it as output, often used for anomaly detection or dimensionality reduction.
6. *Sensor*: A device that detects and measures physical properties (such as temperature, heart rate, or oxygen level) and converts them into signals that can be read and processed by electronic systems.

7. *Cloud (Cloud Computing)*: A technology that allows data and applications to be stored, managed, and accessed remotely over the internet rather than on local servers or personal computers, enabling scalability and real-time access to information.
8. *Triage Algorithm*: A set of rules or a machine learning model used to automatically assess patient data and determine the urgency level of their condition.

## **CHAPTER TWO**

### **LITERATURE REVIEW**

#### **2.1 Conceptual Framework**

Medical triage, the structured process of prioritizing patient care based on clinical urgency, has undergone significant transformation, evolving from battlefield necessity to a highly digitized, intelligent system in modern emergency medicine (Hinson et al., 2025). The conceptual evolution of triage is deeply intertwined with both historical necessity and technological innovation, culminating in today's AI-enhanced, data-driven frameworks (Iserson & Moskop, 2007).

##### **2.1.1 Historical and Developmental trends in Medical Triage**

The origin of triage can be traced back to the Napoleonic Wars, where Dominique Jean Larrey, a French military surgeon, introduced the concept of sorting battlefield casualties to optimize scarce medical resources (Iserson & Moskop, 2007). Larrey's classification divided patients into three key categories: those who will survive without immediate care, those who need immediate intervention, and those unlikely to survive even with care. This principle of prioritizing care to maximize survival outcomes became a cornerstone of military and, later, civilian emergency medicine.

Throughout the 20th century, triage was refined through large-scale conflicts such as World Wars I and II and adapted into civilian emergency departments (EDs). The Manchester Triage System (MTS), Australasian Triage Scale (ATS), Canadian Triage and Acuity Scale (CTAS), and Emergency Severity Index (ESI) became global standards for evaluating and prioritizing patients based on predefined symptom severity categories and resource needs (Khosravi et al., 2024; Määttä et al., 2023; Porto, 2024).

##### **2.1.2 General Concepts and Modern Approaches**

Modern triage integrates clinical decision-making, resource allocation, and increasingly, predictive analytics. Traditional methods relied on clinician judgment, guided by tools like the Glasgow Coma Scale or red flag indicators. However, these approaches often suffered from inter-observer variability, bias, and inefficiency during high patient loads (Challen et al., 2019).

To address this, Artificial Intelligence (AI) has been introduced into triage systems to enhance consistency, speed, and accuracy. AI-driven triage systems utilize machine learning (ML), natural language processing (NLP), and large language models (LLMs) to assess patient severity, predict outcomes, and streamline care pathways (Yi et al., 2025). Techniques such as decision trees, support vector machines, random forests, and XGBoost classifiers have demonstrated high predictive power in both structured (vitals, labs) and unstructured (EHRs, referral letters) data (Annarumma et al., 2019). In recent years, triage has further evolved into IoT-enhanced, remote, and smart-home-based systems, where wearable sensors capture real-time physiological signals and feed them into cloud-based ML models (Saad Mohsin et al., 2025). This is particularly useful for telemedicine and pre-hospital care where early triage decisions can significantly affect outcomes

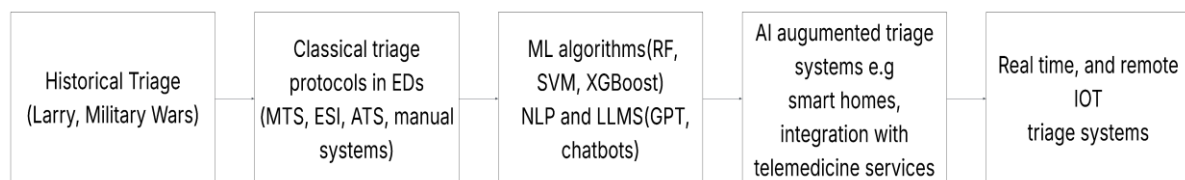


Figure 2.1: Conceptual framework illustrating the evolution of medical triage systems from historical roots to AI-enhanced, IoT-integrated models.

## 2.2 Theoretical Foundations

The theoretical foundation of this study lies at the intersection of clinical decision science, healthcare informatics, and artificial intelligence. Triage systems, especially in emergency medicine, are grounded in theories of urgency prioritization, resource optimization, and human-computer decision support. As the medical field continues to integrate intelligent systems into clinical workflows, several theoretical models and frameworks have emerged to guide their development and evaluation.

### 2.2.1 Utility and Prioritization Theory

One of the main ideas behind triage is the fair and smart use of limited medical resources. This idea comes from utilitarian theory, which means doing the greatest good for the greatest number of people. In emergencies, doctors and nurses must quickly decide who needs help the most so that lives can be saved and time is not wasted (Iserson & Moskop, 2007).



This is the same idea used in early military triage, where doctors decide, which soldiers could return to the battlefield quickly, which ones needed care immediately, and which ones were too badly injured to be helped (Iserson & Moskop, 2007) This way of thinking is still used in today's emergency departments

### **2.2.2 Clinical Decision Support Systems (CDSS)**

As technology improved, Clinical Decision Support Systems (CDSS) were developed to help doctors make better decisions. These systems take in patient information and give suggestions based on data and medical knowledge (Khosravi et al., 2024; Määttä et al., 2023; Yi et al., 2025). They are not meant to replace doctors but to support them. CDSS theory supports the use of AI in triage, especially when it comes to:

- i. Looking at patient data (like vital signs),
- ii. Matching it with what is already known in medicine,
- iii. And helping health workers know what to do next

### **2.2.3 Signal Detection and Pattern Recognition**

Another important idea is from signal detection theory, which explains how people (or machines) can tell important information apart from background noise. In hospitals, this means knowing which symptoms are signs of real danger and those that are not (Ong et al., n.d.).

AI models like XGBoost or Random Forests are built to do just that. They look for patterns in large amounts of data something humans can miss when things are busy (Yi et al., 2025).

### **2.2.4 Human-Technology Interaction**

Finally, socio-technical systems theory explains how people and technology must work together (Carayon et al., 2006). A triage system is not useful if doctors or nurses don't trust it, don't understand it, or can't use it easily. This is why recent triage systems use tools like SHAP values or LIME, which help explain how AI makes decision (Khosravi et al., 2024; Määttä et al., 2023; Porto, 2024). This makes the system easier to understand and more trustworthy for health workers

## **2.3 Empirical Framework**

This section looks at what other researchers have done and found when studying triage systems, especially those using artificial intelligence (AI) and machine learning (ML).

### **2.3.1 Findings from Previous Studies**

Back in 2019, (Annarumma et al., 2019) sought to develop and validate an AI-based triaging system utilizing deep convolutional neural networks for automated prioritization of chest radiographs by clinical urgency. Using 470,388 anonymized radiographs, they employed a two-stage pipeline of Natural Language Processing (NLP) derived priority labels and Convolutional Neural Network (CNN)-based classification, achieving strong specificity (95%) and negative predictive value (99%) but only moderate sensitivity (71%) for critical cases. While the large dataset and workflow simulations demonstrated practical feasibility and potential time savings, the study was limited by single-institution derivation, imperfect ground-truth labels, and relatively poor sensitivity that risked false negatives in urgent cases.

In a 2020 study, (Feng et al., 2020) developed and validated an AI-assisted diagnostic aid for early triage of suspected COVID-19 pneumonia in fever clinics without Computed Tomography (CT) imaging. Using retrospective data from 132 development and 32 validation patients, they applied the Least Absolute Shrinkage and Selection Operator (LASSO) logistic regression to select 18 clinical features, achieving Area Under the Curve (AUC) values of 0.841 and 0.938 with perfect sensitivity but modest specificity (72–78%). The strength lay in its practical focus on resource-limited pandemic settings, but limitations included small single-center samples, retrospective design, and risk of over-allocation due to false positives.

(Sutham et al., 2020) also in their 2020 study designed a mobile triage application for Thailand's pre-hospital emergency system, embedding Criteria Based Dispatch protocols into a smartphone app. Built with the Ionic Framework and tested by 12 emergency staff, the app produced dispatch codes with comparable accuracy to handbook protocols and faster performance in non-trauma cases, though it lagged behind experienced staff for trauma triage. Its strengths were usability-driven development and potential to reduce ED overcrowding, but limitations included small sample size, usability issues, and lack of large-scale clinical validation.

Moving to 2023, (Vântu et al., 2023) conducted a scoping review to map existing evidence on AI in medical triage. Synthesizing studies across emergency, primary, and remote care, they found that AI improved predictive accuracy and speed but faced challenges including inconsistent validation, limited transparency, and few real-world effectiveness studies. The paper's breadth and synthesis provided valuable groundwork, though absence of meta-analysis and limited appraisal of study quality constrained the robustness of its conclusions.

Also in 2023, (Townsend et al., 2023) explored practitioner perspectives on DAISY, an AI-supported triage system in development for the National Health Service (NHS) emergency departments. Through interviews with nine healthcare practitioners, analysed thematically and supplemented by NLP-based sentiment analysis, they found generally positive attitudes, citing reduced wait times and improved diagnostic consistency. While pioneering in its focus on staff perspectives and ethical-cultural considerations, the study was limited by its small sample, reliance on hypothetical scenarios, and single-country scope.

By 2024, (Porto, 2024) conducted a systematic review of AI methods in Emergency Department (ED) triage, focusing particularly on integrating structured clinical data with unstructured free-text notes. Following PRISMA guidelines, the review included 60 studies, finding that logistic regression was most common, while gradient boosting and deep learning achieved superior performance, especially when combined with NLP of triage notes. The strengths were its comprehensive coverage, systematic bias assessment, and novel focus on textual data, but limitations included high methodological heterogeneity, widespread risk of bias, lack of meta-analysis, and limited explainability across reviewed models.

In early 2025, (Hinson et al., 2025) reported on a multisite real-world implementation of the AI-powered tool TriageGO in U.S. emergency departments, aiming to test its impact on triage equity. Analysing 174,648 visits pre- and post-implementation, they found improvements in both sensitivity (79%→83%) and specificity (74%→77%) for identifying critical illness, with the largest gains among marginalized groups, reducing inequities. The study's strength was its pragmatic design and large diverse sample, while limitations were its nonrandomized single-system scope and limited external validity.

Around the same time, (Yi et al., 2025). systematically reviewed prospective studies of AI-based ED triage systems. Across seven cohort studies published between 2017–2023, they found accuracy ranging from 80%–99%, with fuzzy logic models performing best. Their

strengths included focusing only on prospective real-world validations and systematic STROBE-based quality assessment, but limitations included the small number of studies, exclusion of highest-acuity cases in some, and reliance on subjective clinical judgments as gold standards.

Concurrently in 2025, (Saad Mohsin et al., 2025). proposed an AI-IoMT framework for remote triage and diagnosis within telemedicine. They designed a layered system where wearable IoMT devices feed patient data to AI models for triage classification, tested on simulated datasets with promising accuracy and low latency. Strengths included innovation in merging IoMT with AI for underserved regions, but weaknesses lay in reliance on simulations rather than clinical trials, absence of patient outcomes, and limited discussion of algorithmic transparency.

Finally, (Kipourgos et al., n.d.) introduced i-TRIAGE, a dual AI system employing both machine learning (J48) and fuzzy logic within the ESI framework, using data from 616 patients. While machine learning achieved variable results (72–95% accuracy across subsystems), fuzzy logic delivered near-perfect performance (99% accuracy, 93% precision, 99% sensitivity/specificity). Its dual-purpose design supporting nurse triage and specialist recommendation was a notable strength, though single-site data, lack of external validation, and potential overfitting with such high accuracy metrics were important limitations.

### **2.3.2 Gaps and Challenges**

1. Limited generalizability due to single-center or geographically restricted datasets, which makes it difficult to apply findings across diverse healthcare settings (Annarumma et al., 2019; Feng et al., 2020; Hinson et al., 2025; Kipourgos et al., n.d.).
2. Small sample sizes in validation studies, reducing statistical power and confidence in the reported outcomes (Feng et al., 2020; Sutham et al., 2020; Townsend et al., 2023; Yi et al., 2025).
3. Retrospective and nonrandomized study designs, which introduce potential bias and limit causal inference (Feng et al., 2020; Hinson et al., 2025).
4. High false-positive rates and suboptimal sensitivity, which risk over-allocation of resources and missed critical cases (Annarumma et al., 2019; Feng et al., 2020).
5. Lack of large-scale, real-world clinical effectiveness studies, with many relying on simulations, prototypes, or limited pilot (Saad Mohsin et al., 2025; Vântu et al., 2023).

6. High methodological heterogeneity and risk of bias across published AI triage models, preventing meta-analysis and weakening evidence quality (Porto, 2024; Vântu et al., 2023).
7. Limited algorithm transparency and explainability, raising concerns about trust and interpretability in clinical use (Porto, 2024; Vântu et al., 2023).
8. Usability and integration challenges, including confusing user interfaces or lack of clear pathways for embedding AI tools into existing workflows (Kipourgos et al., n.d.; Sutham et al., 2020).
9. Exclusion of the most critical triage levels (1–2) in some validations, reducing applicability to life-threatening emergencies (Yi et al., 2025)
10. Overfitting and inflated performance metrics in certain models, particularly fuzzy logic systems without external validation, which undermine confidence in reported accuracy (Kipourgos et al., n.d.)

### **2.3.3 Research Gap for This Study**

The collective gap evident across existing studies is that, although AI has demonstrated promise in enhancing triage accuracy and efficiency, most approaches remain constrained by retrospective or single-center designs, limited external validation, lack of workflow integration, and minimal applicability to remote or resource-limited contexts; in contrast, the present study advances the field by developing a cloud-based, AI-enabled triage system integrated with IoT wearables, thereby enabling real-time, automated, and scalable triage support that is both hospital-ready and adaptable to local and remote healthcare environments.

### **2.4 Summary of Literature Review**

This chapter reviewed the key ideas, theories, and findings related to medical triage, especially how it is being improved using artificial intelligence (AI) and machine learning (ML). In the Conceptual Framework, we looked at how triage started on the battlefield and has grown into a structured process used in hospitals. Over time, it has moved from being done entirely by humans to being supported by smart technologies, like AI, wearable devices, and online tools. A visual diagram was also used to show how triage systems have evolved over time.

The research further discussed several models and theories that explain how triage works. These included theories about making the best use of limited resources, helping doctors make

decisions, separating urgent from non-urgent cases, and making sure that people and machines can work well together. These theories help explain why AI tools are useful in triage.

Furthermore, the research the works of other researchers in the field. Many studies show that AI can make triage faster, more accurate, and more consistent than when done by humans alone. However, we also saw that there are still challenges. These include problems with poor-quality data, unfair predictions, difficulty in understanding AI decisions, and gaps in research about real-time or remote triage.

In summary, literature shows that AI-based triage is promising although yet not perfect, with potentials to improve patient care, reduce waiting times, and make better use of healthcare resources. However, to fully reach this potential, more research is needed especially on how to combine different AI tools and how to make these systems easy to use, trustworthy, and fair.

## **CHAPTER THREE**

### **RESEARCH METHODOLOGY**

#### **3.0 Choice of Methodology**

The development of this system employed the Prototyping Model and the Structured Systems Analysis and Design Methodology (SSADM) because of their suitability for systems requiring both iterative validation and structured analysis. The Prototyping Model was chosen due to the experimental nature of integrating IoT sensors, live data streams, and AI-based triage decision making, allowing early testing and refinement of modules such as vital sign monitoring, data transmission, and acuity computation before full deployment. Meanwhile, SSADM provided a systematic framework for analysing data flows, defining system processes, and ensuring logical consistency between the hardware, software, and AI components. Together, these methodologies ensured that the system was both technically robust and adaptable capable of evolving through testing while maintaining clear structural and functional integrity.

#### **3.1 Data Collection**

A typical hospital triage process requires a concise set of structured and unstructured data elements to enable rapid clinical prioritization. Core data include patient identifiers (name, age, sex, unique patient/triage ID), presenting complaint and symptom description (free text or structured fields), and past medical history (chronic conditions, allergies, current medications). Immediate objective measurements are essential: vital signs (temperature, heart rate, respiratory rate, blood pressure) and point-of-care test results (e.g., glucose, ECG where available). Operational and contextual metadata ie time stamps, location/arrival mode, staff observations, and prior encounter records support decision timing and continuity of care. Finally, the process requires triage category/score outputs and secure audit logs/documentation (records of reassessments, notifications and actions) to guide treatment and enable retrospective review.

## Conventional Triage Process

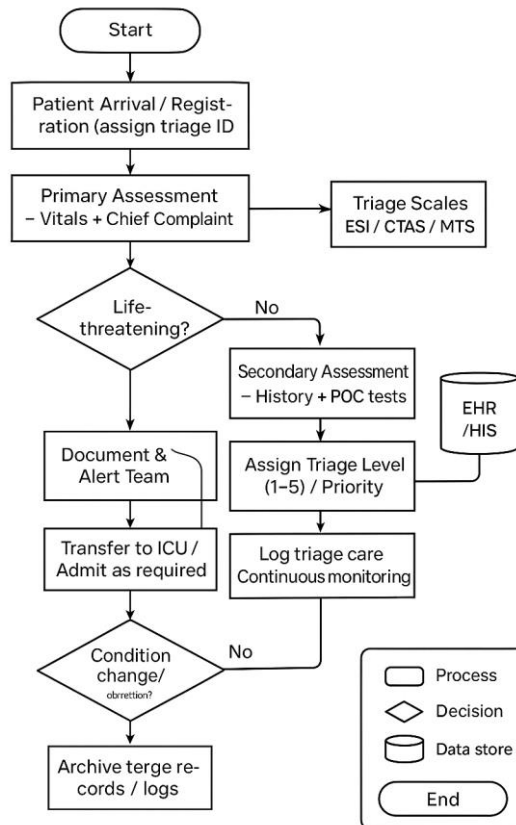


Figure 3.1 Conventional Triage Process

The conventional triage process in hospital settings follows a structured workflow designed to ensure that patients are assessed and prioritized efficiently upon arrival. The conventional triage workflow begins at patient arrival and registration, where administrative data and a unique triage identifier are recorded to establish the patient's record. Immediately following registration, a primary assessment is performed: triage staff collect complaints and measure core vital signs (temperature, heart rate, respiratory rate, blood pressure, SpO<sub>2</sub> and level of consciousness). These initial data provide the objective basis for urgent decision-making.

Next, the clinician conducts a focused secondary assessment that integrates the patient's brief history, point-of-care test results (where available), and clinical observations. This helps to determine whether the case is life-threatening; patients identified as such proceed directly to immediate resuscitation and stabilization. Non-life-threatening cases are assigned a formal triage category (e.g., critical/urgent/non-urgent or Level 1–5) using institutional triage criteria, and this classification is recorded in the patient's chart and the hospital information system.

Following categorization, patients are directed to the appropriate care stream (resuscitation bay, emergency treatment area, observation, or outpatient care). Those awaiting treatment remain



under continuous monitoring and are re-assessed at defined intervals; any deterioration triggers an immediate re-triage and escalation loop back to the assessment phase. Throughout the process all assessments, timestamps, notifications, and disposition decisions are documented to create an auditable trail for clinical handover and quality review.

In this study, patient vital signs were acquired using IoT-based sensors interfaced with the ESP32 NodeMCU microcontroller. The primary physiological parameter measured directly was body temperature, obtained through the LM35 temperature sensor. However, due to hardware limitations and calibration challenges associated with the blood pressure (BP) sensor module, blood pressure data was simulated to reflect realistic patient readings. This simulation ensured the system's ability to process, analyse, and prioritize patient conditions under near real world scenarios while maintaining continuous data flow within the Kafka streaming architecture. The simulated BP values were generated within medically accepted physiological ranges, allowing effective testing of the acuity computation and criticality ranking algorithms.

### **3.2. Prototype Design**

The input/output (I/O) design of the medical triage system focuses on establishing the physical connection between the IoT hardware components and the microcontroller interface to enable accurate data acquisition and transmission. The system uses an ESP8266 NodeMCU (Wemos D1 Mini) as the primary processing and communication unit due to its built-in Wi-Fi capability and single analog-to-digital converter (ADC) input labelled A0 (ADC0). The key hardware components required for this setup include:

- ESP8266 NodeMCU / Wemos D1 Mini: serves as the central microcontroller responsible for reading sensor data and transmitting it to the cloud via Wi-Fi.
- LM35 Temperature Sensor (3-pin analog output): used to measure patient body temperature in real time.
- Small Protoboard: provides a stable base for circuit assembly.
- Jumper Wires and Connectors: facilitate electrical connections between components.
- Resistors (optional): used when a voltage divider is required for boards with ADC input scales greater than 1V (though some NodeMCU boards handle this internally).

The LM35 sensor was connected following its standard pin configuration, with the Vcc pin (left) linked to the 3.3V output of the NodeMCU, the Vout pin (middle) connected to A0 (ADC0) for analog signal reading, and the GND pin (right) connected to the ground (GND) terminal of the NodeMCU. The LM35 produces a linear output of 10 mV per °C, allowing the ESP8266 to read precise temperature variations, which are then transmitted securely via HTTPS to the system's Kafka-based data pipeline for triage processing.

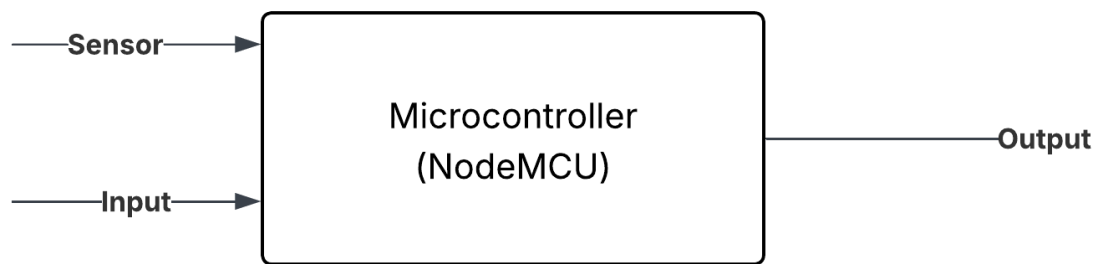


Figure 3.2 Prototype Design

### 3.3. System Architecture

The system architecture of the proposed Medical Triage System is designed to facilitate real time patient monitoring, data acquisition, intelligent analysis, and communication between patients and healthcare professionals. The architecture follows a layered, data-driven flow that begins at the patient level and culminates in informed decision-making by medical doctors.

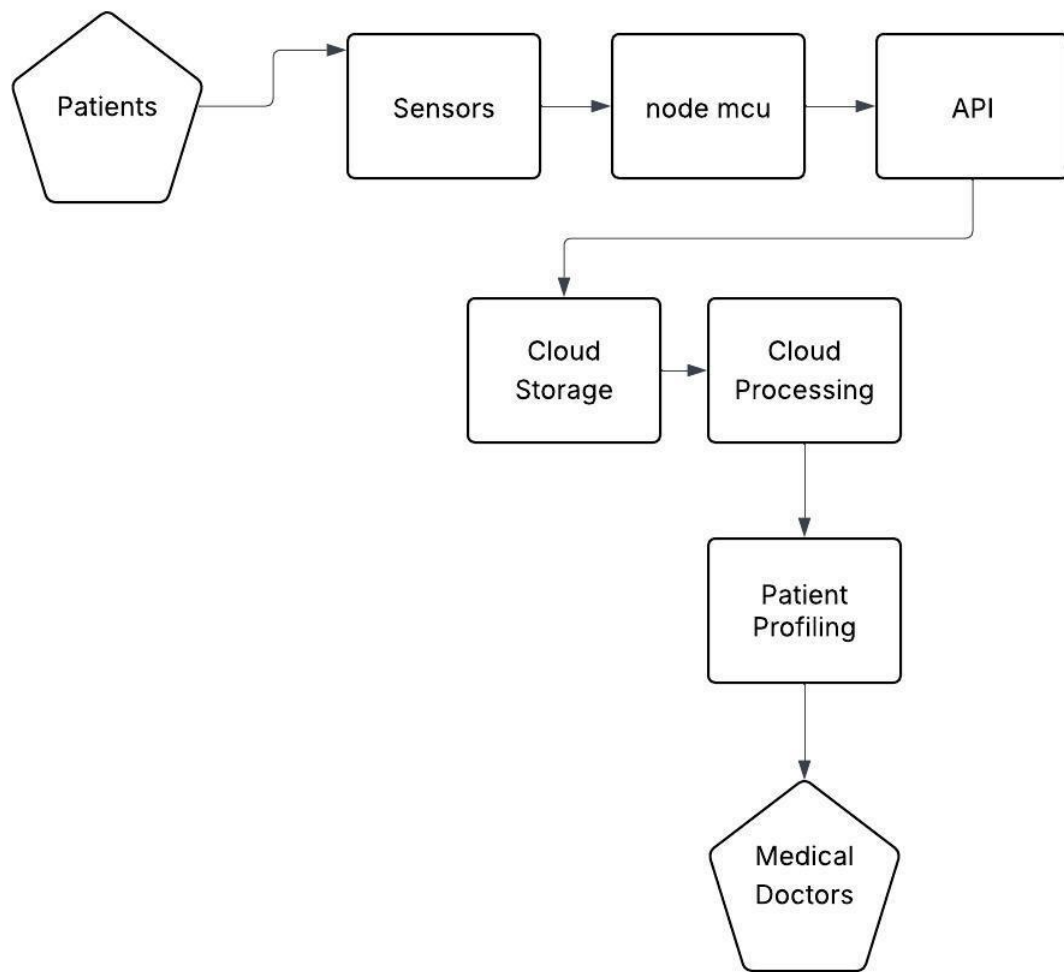


Fig 3.3 Architectural diagram for the system

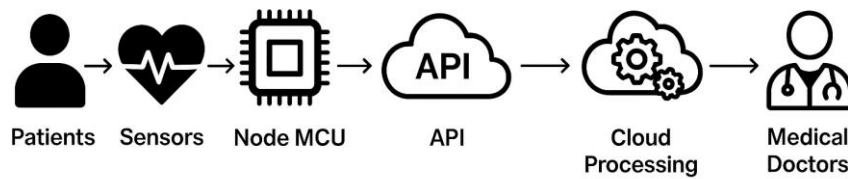


Fig 3.4 visualization of the system architecture

### 3.3.1 Overview

The architecture, as illustrated in Figure 3.3, consists of seven major components: Patients, Sensors, NodeMCU, API Gateway, Cloud Storage, Cloud Processing, Patient Profiling, and Medical Doctors. Each component plays a critical role in ensuring efficient data flow, security, and reliability within the triage process.

#### System Flow Description

1. **Patients:** Patients represent the origin of physiological and contextual data in the system. They are equipped with non-invasive biomedical sensors capable of capturing key health indicators such as heart rate, temperature, oxygen saturation (SpO<sub>2</sub>), and blood pressure.
2. **Sensors:** The sensors form the first technical layer of the architecture. They continuously measure physiological parameters and convert them into digital signals. These sensors are strategically placed to ensure accurate and real-time data capture.
3. **NodeMCU (Data Acquisition and Transmission Unit):** The NodeMCU microcontroller acts as an intermediary between the sensors and the cloud infrastructure. It collects data from the sensors via analog or digital interfaces, preprocesses the readings (e.g., normalization and filtering), and transmits the data over Wi-Fi using HTTP or MQTT protocols to the API layer.
4. **API Gateway:** The API serves as a secure communication channel between the IoT edge device and the cloud backend. It receives incoming data streams from the NodeMCU, validates and formats them, and then forwards the data to the appropriate

cloud storage service. This layer also enforces authentication, rate-limiting, and data integrity.

5. **Cloud Storage:** The cloud storage module is responsible for persistently storing the incoming medical data. It supports structured and semi-structured formats, ensuring that both raw sensor data and processed health metrics are archived for future retrieval and analytics. Examples of cloud storage solutions include Firebase, AWS S3, or Google Cloud Storage.
6. **Cloud Processing (Analytics and Decision Engine):** This component hosts the core intelligence of the system. It executes triage algorithms, anomaly detection models, and priority classification logic to determine the urgency of patient conditions. Machine learning or rule-based models may be deployed to classify cases into acuity levels such as *critical*, *urgent*, or *stable*.
7. **Patient Profiling:** The patient profiling subsystem aggregates and contextualizes all collected data to build comprehensive health profiles. It combines historical records, demographic data, and recent sensor readings to provide doctors with actionable insights into each patient's health status and risk trends.
8. **Medical Doctors (End-User Layer):** Medical professionals access the processed and profiled data through a secure web or mobile dashboard. The interface presents real-time health metrics, triage scores, and alerts, enabling doctors to prioritize cases and make timely medical interventions.

### **3.4. Profiling Algorithm**

#### **3.4.1. Overview**

The proposed algorithm provides an intelligent triage acuity profiling and scheduling system that dynamically assesses patient urgency by correlating natural language summaries of clinical data (`nlp_summary`) with live physiological sensor readings (temperature and other vitals). The goal is to validate and quantify the reliability of initially collected clinical information by comparing it with real-time vital signs, ensuring that patient priority is assigned based on both semantic context and physiological evidence.

### 3.4.2. Input Data Streams

1. NLP Summary (nlp\_summary)
  - a. Generated from the structured or unstructured clinical data collected during intake (e.g., symptoms, complaints, triage notes).
  - b. Encodes patient state in a semantic embedding vector  $\text{snlp} \in \mathbb{R}^d$ .
  - c. Each element represents a latent medical feature (e.g., distress level, fever indication, cardiovascular symptoms).
2. Live Vital Signs (vitals)
  - a. Streamed physiological measurements from sensors.
  - b. Current implementation includes:
    - i. Temperature  $T_t$  (in  $^{\circ}\text{C}$ )
    - ii. Heart Rate / Sensor Signal  $H_t$  (in bpm or arbitrary normalized sensor units)
  - d. Each measurement is timestamped  $t$  to ensure temporal synchronization with the NLP summary context.

### 3.4.3. Processing Time and Synchronization

When new vital data is received (e.g.,  $T_t$ ,  $H_t$ ), the system logs the timestamp  $\tau_v = \text{datetime.now}()$ . The NLP summary has its own creation timestamp  $\tau_n$ . A synchronization window  $\Delta\tau = |\tau_v - \tau_n| \leq \epsilon$  (e.g., 5 minutes) ensures the NLP context remains valid for comparison. If  $\Delta\tau > \epsilon$ , the algorithm re-requests or reprocesses the NLP summary to reflect the most current state, maintaining temporal integrity.

### 3.4.4. Mathematical Framework

The acuity score computation proceeds through three main components:

- a) NLP-Vitals Correlation Mapping: The NLP embedding vector  $\text{snlp}$  is mapped into a physiological expectation space using a learned linear or nonlinear transformation  $f_{\theta}$

$$v = f_{\theta}(\text{snlp})$$

where:

$v = [T, H]$  represents the expected vital signature inferred from the textual clinical summary.

- b) Deviation Analysis: The live vitals  $v_t=[T_t, H_t]$  are compared against the predicted vitals  $\hat{v}$ :

$$\Delta v = |v_t - \hat{v}|$$

A weighted deviation score  $D$  quantifies how much the physiological data deviates from the linguistic prediction:

$$D = w_T \cdot \sigma_T |T^\wedge - T_t| + w_H \cdot \sigma_H |H^\wedge - H_t|$$

where  $\sigma_T$  and  $\sigma_H$  are normalization constants (e.g., standard deviations from baseline population data), and  $w_T, w_H$  are clinician-tuned weights (e.g., temperature deviations are more critical in infectious triage).

- c) Reliability and Acuity Scoring: The reliability score  $R$  expresses the degree of alignment between NLP-derived expectations and sensor reality:

$$R = e^{-\alpha D}$$

where  $\alpha$  controls sensitivity (higher  $\alpha \rightarrow$  lower tolerance for discrepancy).

The acuity score  $A$  integrates both the clinical severity inferred from the NLP summary ( $S$ ) and the reliability measure  $R$ :

$$A = \beta_1 S + \beta_2 (1 - R)$$

$S$ : Severity derived from NLP classification (e.g., 0–1 scale from a triage classifier)

$R$ : NLP–vitals reliability score

$\beta_1, \beta_2$ : weight coefficients ensuring balanced influence of text and vitals

Higher  $A$  indicates greater acuity and higher scheduling priority.

### 3.4.5. Scheduling and Decision Logic

Based on  $A$ , the patient is assigned a triage level or priority schedule slot:

| Acuity Score $A$   | Triage Level | Scheduling Action                  |
|--------------------|--------------|------------------------------------|
| $A \geq 0.8$       | Critical     | Immediate physician attention      |
| $0.5 \leq A < 0.8$ | Urgent       | Prioritized queue                  |
| $0.2 \leq A < 0.5$ | Routine      | Normal scheduling                  |
| $A < 0.2$          | Low Priority | Deferred follow-up or remote check |

### 3.4.6. System Operation Flow

- Data Collection: Patient fills intake form; NLP engine summarizes into `nlp_summary`.

- Vital Acquisition: Live temperature and sensor readings are captured and timestamped.
- Vector Correlation: NLP embeddings compared with live vitals via correlation mapping.
- Scoring: Reliability and acuity scores are computed.
- Decision Layer: Scheduler assigns triage priority and queue placement.

### 3.5 Evaluation

A compact results table (rounded where appropriate) followed by **full, digit-by-digit calculations** for each case.

Summary table of simulated experiments

| Case | Short description.                             | S (NLP) | T <sup>^</sup> (°C) | T (°C) | P <sup>^</sup> | P   | D      | R=e-D    | A              | Model triage | Ground truth (simulated) | Correct ? |
|------|------------------------------------------------|---------|---------------------|--------|----------------|-----|--------|----------|----------------|--------------|--------------------------|-----------|
| C1   | Concordant normal                              | 0.30    | 37.0                | 37.2   | 101.3          | 100 | 0.2660 | 0.766439 | 0.28007        | Routine      | Routine                  | ✓         |
| C2   | Extreme high T & P concordant                  | 0.95    | 40.0                | 41.5   | 250            | 260 | 2.0000 | 0.135335 | <b>0.92440</b> | Critical     | Critical                 | ✓         |
| C3   | NLP says no fever, vitals high (NLP FN)        | 0.10    | 36.7                | 39.0   | 100            | 105 | 2.8600 | 0.057269 | 0.35282        | Routine      | <b>Critical</b>          | ✗ (F)     |
| C4   | NLP predicts severe but vitals normal (NLP FP) | 0.90    | 39.5                | 37.0   | 180            | 102 | 4.5600 | 0.010462 | <b>0.92686</b> | Critical     | <b>Low</b>               | ✗ (F)     |
| C5   | Sensor noisy (small noise)                     | 0.60    | 38.0                | 38.3   | 120            | 125 | 0.4600 | 0.631284 | 0.53061        | Urgent       | Urgent                   | ✓         |
| C6   | Stale vitals (beyond sync window)              | 0.70    | 38.0                | 37.5   | 130            | 128 | 0.6400 | 0.527292 | 0.63181        | Urgent       | Urgent                   | ✓ *       |



| Case | Short description.                               | S (NLP) | T <sup>^</sup> (°C) | T (°C) | P <sup>^</sup> | P   | D       | R=e-D      | A       | Model triage | Ground truth (simulated) | Corre ? |
|------|--------------------------------------------------|---------|---------------------|--------|----------------|-----|---------|------------|---------|--------------|--------------------------|---------|
|      |                                                  |         |                     |        |                |     |         |            |         |              |                          |         |
| C7   | Missing pressure sensor                          | 0.50    | 37.8                | 38.1   | 110            | —   | 0.3600* | 0.697676   | 0.44070 | Routine      | Routine                  | ✓       |
| C8   | Extreme discordance: vitals critical but NLP low | 0.05    | 37.0                | 42.0   | 100            | 300 | 10.0000 | 0.00004540 | 0.33499 | Routine      | <b>Critical</b>          | ✗ (F    |

For C7 we renormalized weights because pressure was missing: effective  $wT' = wT/(wT + wP) = 0.6/1.0 = 0.6$  and  $wP' = 0$ . Calculation notes follow.

Full step-by-step numeric derivations (digit-by-digit arithmetic)

Case C1: Concordant normal

Inputs:

1.  $S=0.30, T^{\wedge}=37.0, T=37.2, P^{\wedge}=101.3, P=100$ .

- $\Delta T = |T^{\wedge} - T| = |37.0 - 37.2| = 0.2$

- $\Delta P = |P^{\wedge} - P| = |101.3 - 100| = 1.3$

- Normalized terms:

- $\sigma T \Delta T = 0.50.2 = 0.4$

- $\sigma P \Delta P = 201.3 = 0.065$

- Weighted sum:

- $wT \cdot 0.4 = 0.6 \times 0.4 = 0.24$

- $wP \cdot 0.065 = 0.4 \times 0.065 = 0.026$

- $D = 0.24 + 0.026 = 0.266$

- Reliability:
  - $R = e^{-D} = e^{-0.266} \approx 0.766439$  (numerical)
- Acuity:
  - $1 - R = 1 - 0.766439 = 0.233561$
  - $\beta_1 S = 0.7 \times 0.30 = 0.21$
  - $\beta_2(1 - R) = 0.3 \times 0.233561 = 0.0700683$
  - $A = 0.21 + 0.0700683 = 0.2800683 \rightarrow \text{Routine}$

Case C2: Extreme high temperature & pressure (concordant)

Inputs:

- $S = 0.95, T^{\wedge} = 40.0, T = 41.5, P^{\wedge} = 250, P = 260$ .
- 1.  $\Delta T = |40.0 - 41.5| = 1.5$
- 2.  $\Delta P = |250 - 260| = 10$
- 3. Normalized:
  - a.  $\Delta T / \sigma T = 1.5 / 0.5 = 3.0$
  - b.  $\Delta P / \sigma P = 10 / 20 = 0.5$
- 4. Weighted:
  - a.  $w_T \cdot 3.0 = 0.6 \times 3.0 = 1.8$
  - b.  $w_P \cdot 0.5 = 0.4 \times 0.5 = 0.2$
  - c.  $D = 1.8 + 0.2 = 2.0$
- 5.  $R = e^{-2.0} \approx 0.135335$
- 6.  $1 - R = 0.8646647$ 
  - a.  $\beta_1 S = 0.7 \times 0.95 = 0.665$
  - b.  $\beta_2(1 - R) = 0.3 \times 0.8646647 = 0.2593994$
  - c.  $A = 0.665 + 0.2593994 = 0.9243994 \rightarrow \text{Critical}$

Interpretation: both text and vitals indicate severe physiology  $\rightarrow$  correct Critical classification.

Case C3 — NLP says no fever but vitals high (NLP false negative)

Inputs:

- 1.  $S = 0.10, T^{\wedge} = 36.7, T = 39.0, P^{\wedge} = 100, P = 105$ .
- 1.  $\Delta T = |36.7 - 39.0| = 2.3$

$$2. \Delta P = |100 - 105| = 5$$

3. Normalized:

$$a. \Delta T / \sigma T = 2.3 / 0.5 = 4.6$$

$$b. \Delta P / \sigma P = 5 / 20 = 0.25$$

4. Weighted:

$$a. wT \cdot 4.6 = 0.6 \times 4.6 = 2.76$$

$$b. wP \cdot 0.25 = 0.4 \times 0.25 = 0.10$$

$$c. D = 2.76 + 0.10 = 2.86$$

$$5. R = e^{-2.86} \approx 0.0572688$$

$$6. 1 - R = 0.9427312$$

$$a. \beta_1 S = 0.7 \times 0.10 = 0.07$$

$$b. \beta_2 (1 - R) = 0.3 \times 0.9427312 = 0.28281936$$

$$c. A = 0.07 + 0.28281936 = 0.35281936 \rightarrow \text{Routine}$$

Interpretation / problem: live vitals look concerning (high temp), but because NLP severity  $S$  is tiny (0.10), the fusion formula gives a low-to-moderate  $A$ . This produces a false negative (undertriage). This shows the fusion formula, as tuned, may not automatically escalate if  $S$  is very small even when vitals are extreme.  $\Rightarrow$  *need deterministic vital thresholds or larger  $\beta_2$  (or clinical override).*

Case C4: NLP predicts severe but vitals normal (false positive)

Inputs:

$$S = 0.90, T^{\wedge} = 39.5, T = 37.0, P^{\wedge} = 180, P = 102.$$

$$\Delta T = |39.5 - 37.0| = 2.5$$

$$\Delta P = |180 - 102| = 78$$

Normalized:

$$\Delta T / \sigma T = 2.5 / 0.5 = 5.0$$

$$\Delta P / \sigma P = 78 / 20 = 3.9$$

Weighted:

$$wT \cdot 5.0 = 0.6 \times 5.0 = 3.0$$

$$wP \cdot 3.9 = 0.4 \times 3.9 = 1.56$$

$$D = 3.0 + 1.56 = 4.56$$

$$R = e^{-4.56} \approx 0.0104621$$

$$1-R \approx 0.9895379$$

$$\beta_1 S = 0.7 \times 0.90 = 0.63$$

$$\beta_2(1-R) = 0.3 \times 0.9895379 = 0.2968614$$

$$A = 0.63 + 0.2968614 = 0.9268614 \rightarrow \text{Critical}$$

Interpretation / problem: NLP suggested severity but vitals are actually normal. The large deviation D (because  $P^\wedge$  predicted very high but measured P is low) produces very small R, so  $(1-R) \approx 0.99$  adds to acuity; because S was already high (0.90), model strongly classifies as Critical  $\rightarrow$  false positive if ground truth is Low. This demonstrates the fusion amplifies discordance  $(1 - R)$  even when the vitals are reassuring — again suggesting you must add sanity checks (e.g., if all vitals in safe ranges, clinician review rather than automatic Critical) or tune weights.

Case C5: Sensor noisy (small noise)

Inputs:

- $S=0.60, T^\wedge=38.0, T=38.3, P^\wedge=120, P=125$ .

1.  $\Delta T=0.3 \rightarrow \Delta T/\sigma T=0.3/0.5=0.6$

2.  $\Delta P=5 \rightarrow \Delta P/\sigma P=5/20=0.25$

3. Weighted:

- $wT \cdot 0.6 = 0.36$

- $wP \cdot 0.25 = 0.1$

- $D = 0.36 + 0.1 = 0.46$

4.  $R = e^{-0.46} \approx 0.6312836$

5.  $1-R = 0.3687164$

- $\beta_1 S = 0.7 \times 0.6 = 0.42$

- $\beta_2(1-R) = 0.3 \times 0.3687164 = 0.1106149$

- $A = 0.42 + 0.1106149 = 0.5306149 \rightarrow \text{Urgent}$

This shows small sensor noise can raise acuity from 0.6 NLP severity to an Urgent classification.

Case C6: Stale vitals beyond sync window

Inputs:

- $S=0.70, T^\wedge=38.0, T=37.5, P^\wedge=130, P=128$ .

- *Note:* This case is flagged stale = True (i.e.,  $\Delta\tau > \epsilon$ ) — we still compute scores but the system must surface a stale flag for clinician review.

1.  $\Delta T=0.5 \rightarrow \Delta T/\sigma T=0.5/0.5=1.0$
2.  $\Delta P=2 \rightarrow \Delta P/\sigma P=2/20=0.1$
3. Weighted:
  - $wT \cdot 1.0=0.6 \times 1.0=0.6$
  - $wP \cdot 0.1=0.4 \times 0.1=0.04$
  - $D=0.6+0.04=0.64$
4.  $R=e^{-0.64} \approx 0.5272924$
5.  $1-R=0.4727076$ 
  - $\beta_1 S=0.7 \times 0.7=0.49$
  - $\beta_2(1-R)=0.3 \times 0.4727076=0.1418123$
  - $A=0.49+0.1418123=0.6318123 \rightarrow \text{Urgent}$

Operational note: because vitals are stale, the score should not be used to auto-escalate without clinician confirmation flag and re-measure.

#### Case C7: Missing pressure sensor

Inputs:

- $S=0.50, T^{\wedge}=37.8, T=38.1, P^{\wedge}=110, P=\text{missing}$ .

Handling: renormalize weights because P is missing. Effective  $wT'=wT/(wT+wP)=0.6/1.0=0.6$  (same here) and  $wP'=0$ .

1.  $\Delta T=|37.8-38.1|=0.3$
2.  $\Delta T/\sigma T=0.3/0.5=0.6$
3. Weighted:  $D=wT' \cdot 0.6=0.6 \times 0.6=0.36$
4.  $R=e^{-0.36} \approx 0.6976763$
5.  $1-R=0.3023237$ 
  - $\beta_1 S=0.7 \times 0.5=0.35$
  - $\beta_2(1-R)=0.3 \times 0.3023237=0.09069711$
  - $A=0.35+0.09069711=0.44069711 \rightarrow \text{Routine}$

**Note:** missing sensor reduces information; the system should lower confidence and ask for re-measure or rely on clinician.

#### Case C8: Extreme discordance: vitals critical but NLP low (dangerous false negative)

Inputs:

- $S=0.05, T^{\wedge}=37.0, T=42.0, P^{\wedge}=100, P=300$ .

1.  $\Delta T = |37.0 - 42.0| = 5.0$
2.  $\Delta P = |100 - 300| = 200$
3. Normalized:
  - $\Delta T / \sigma T = 5.0 / 0.5 = 10.0$
  - $\Delta P / \sigma P = 200 / 20 = 10.0$
4. Weighted:
  - $w_T \cdot 10.0 = 0.6 \times 10.0 = 6.0$
  - $w_P \cdot 10.0 = 0.4 \times 10.0 = 4.0$
  - $D = 6.0 + 4.0 = 10.0$
5.  $R = e^{-10} \approx 0.00004539993$  (very close to zero)
6.  $1 - R \approx 0.9999546$ 
  - $\beta_1 S = 0.7 \times 0.05 = 0.035$
  - $\beta_2 (1 - R) = 0.3 \times 0.9999546 = 0.2999864$
  - $A = 0.035 + 0.2999864 = 0.3349864 \rightarrow \text{Routine}$

Crucial interpretation: although vitals are *extremely* abnormal ( $T=42^\circ\text{C}$ ,  $P=300$ ), the algorithm still returns a Routine acuity score because the NLP severity  $S$  was very small ( $0.05$ ), and  $\beta_2$  ( $0.3$ ) is not large enough to let the discordance term alone push  $A$  into Critical. This is a dangerous false negative and demonstrates why deterministic safety overrides are essential: extremely abnormal vitals must trigger immediate escalation regardless of NLP.

### **Analysis of misclassifications (false positives / false negatives)**

- **False Negatives (undertriage):**
  - C3 and C8: both show the same root cause: NLP severity  $S$  is very low, and the fusion weights give too much absolute influence to  $S$ . Even massive vitals discordance (C8) did not push  $A$  high enough.
  - Remedies: (a) add deterministic thresholds on vitals (e.g., if  $T \geq 41^\circ\text{C}$  or  $P >$  critical, immediately escalate regardless of  $A$ ), (b) increase  $\beta_2$  or adjust fusion to include a max rule  $A = \max(A, f_{\text{vitals}}(v))$ , (c) use safety-layer classifier that checks absolute vital ranges first.
- **False Positives (overtriage):**

- C4: NLP strongly predicted severity but vitals were normal. The fusion amplified discordance and produced Critical.
- Remedies: include clinical rules that require at least one concordant objective sign (or a deterministic block that if all vitals are within normal safe ranges and no critical thresholds tripped, then escalate to clinician review rather than immediate Critical).
- **Operational concerns:**
  - Stale data (C6): flags must prevent automatic decision-making; require remeasurement or clinician confirmation.
  - Missing data (C7): system should degrade gracefully and ask for remediation.

### **Recommended improvements & safe design patterns (practical changes)**

#### 1. Deterministic safety overrides (must-haves):

- If any vital exceeds absolute emergency thresholds (e.g.,  $T \geq 41^\circ\text{C}$ , pressure  $P >$  device-defined critical), immediately assign Critical and alert clinicians bypass or override fusion. This fixes C8.

#### 2. Hybrid fusion rule:

- Keep the probabilistic fusion A but also compute a vitals-only emergency score V.
- Let final decision be:

decision in emergency zone use fused Final decision = {Critical else use fused A  
if V in emergency zone

#### 3. Tuning / learning:

- Learn  $\beta$  and  $\alpha$  on labelled data; consider making  $\beta_2$  larger if vitals should have equal or larger influence.

#### 4. Clinician-in-the-loop:

- For high discordance (D above a threshold), route to immediate clinician review rather than automatic high-priority scheduling.

#### 5. Explicit stale / missing data handling:

- If  $\Delta\tau > \epsilon$ , set stale flag and do not auto-escalate; require re-measure or manual override.

## **CHAPTER FOUR**

### **SYSTEM DESIGN AND IMPLEMENTATION**

This chapter presents a comprehensive overview of the system design and implementation for the Development of a Cloud-Based IoT and Machine Learning System for Intelligent Medical Triage and Patient Profiling. It highlights the system requirements, architectural framework, and various design diagrams such as use case, activity, class, sequence, and database models. Additionally, it covers the specifics of the implementation, testing strategies, and obstacles faced throughout the development phase. This thorough documentation guarantees that the system's features are consistent with the project's goals.

#### **4.1 System Overview**

The Cloud-Based IoT and Machine Learning System for Intelligent Medical Triage and Patient Profiling is designed to automate and improve the medical triage process using real-time physiological data. The system integrates IoT wearable devices, machine learning algorithms, and cloud computing to collect, process, and analyze patient information for accurate triage classification. The system works by gathering vital signs such as body temperature, heart rate, and oxygen saturation from connected IoT devices. These data are transmitted to the cloud, where a trained machine learning model evaluates patient condition and assigns a triage level such as critical, urgent, or non-urgent. The classified results are displayed through a simple web-based interface for both healthcare workers and patients.

Unlike traditional triage systems that rely on manual judgment, this solution enables faster and more objective decision-making, especially in remote and low-resource regions where medical personnel are limited. It also maintains individual patient profiles for continuous monitoring and future reference.

In summary, the system combines automation, real-time data processing, and intelligent analysis to enhance the accuracy, speed, and accessibility of medical triage services.



## 4.2 Prototype Development

The temperature monitoring system prototype was designed using the LM35 precision temperature sensor integrated with the ESP8266 NodeMCU microcontroller. The LM35 was selected for its linear output voltage proportional to temperature ( $10\text{mV}/^{\circ}\text{C}$ ), providing superior accuracy compared to conventional thermistors. The system operates on a 3.3V power supply, with the sensor's analog output connected to the NodeMCU's ADC pin (A0) for real-time temperature data acquisition. The simulation phase involved calculating the conversion factor for analog-to-digital conversion. Since the NodeMCU's ADC has a 10-bit resolution (0-1023) spanning 3.3V, the raw sensor values are scaled accordingly. The conversion algorithm divides the raw ADC reading by 1024 and multiplies by 3300mV to obtain millivolt values, which are then divided by 10 to yield temperature in degrees Celsius.

### 4.2.1 Hardware Components

- NodeMCU ESP8266 (ESP-12E Module)
- LM35 Precision Temperature Sensor
- Breadboard
- Jumper Wires (Male-to-Male)
- Micro USB Cable

### 4.2.2 Software Components

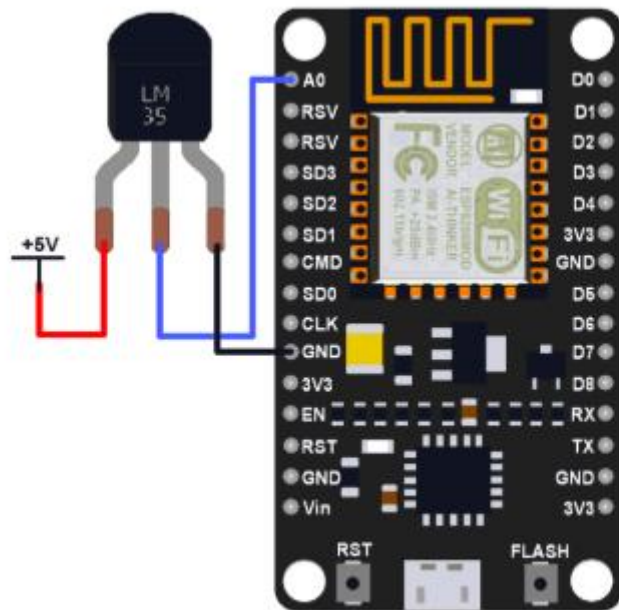
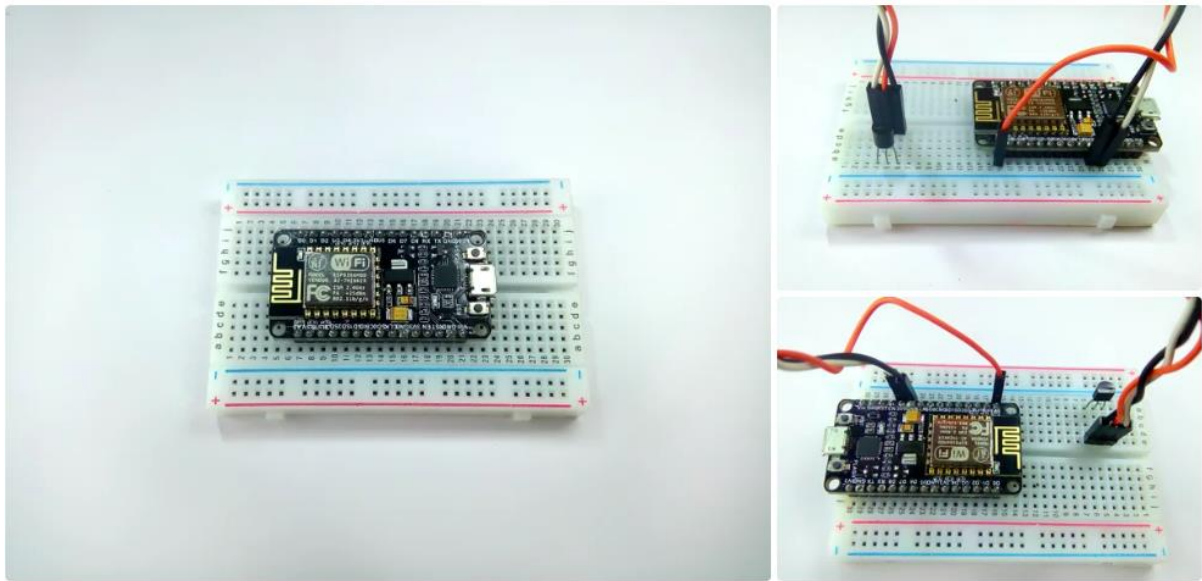
- Arduino IDE (with ESP8266 board support)
- Custom firmware for temperature acquisition and serial monitoring

### 4.2.3 Circuit Implementation

The hardware connections were established as follows:

- Pin 1 (VCC): Connected to NodeMCU 3.3V output
- Pin 2 (VOUT): Connected to NodeMCU analog pin A0
- Pin 3 (GND): Connected to NodeMCU ground

The firmware was developed in Arduino IDE with a sampling interval of 1000ms. The analog readings undergo mathematical processing: the raw ADC value is converted to millivolts using the formula  $(\text{analogValue}/1024.0) \times 3300$ , and subsequently to Celsius through division by 10. The system also provides Fahrenheit conversion using the standard formula  $((^{\circ}\text{C} \times 9)/5) + 32$ .



NodeMCU Interfacing with LM35

**Figure 4.1:** Temperature Monitoring System Circuit Diagram

### 4.3 Test-led Arrangement

Controlled testing was conducted to evaluate the system's performance under varying temperature and simulated pressure conditions. The test environment was configured using a heat source (soldering iron/heat gun) and an insulated chamber to create controlled temperature gradients ranging from 25°C to 100°C. Pressure simulation was achieved through thermal expansion principles within the sealed test chamber.

Test Parameters:

- Temperature Range: 25°C - 100°C (ambient to high temperature)
- Test Duration: 30 minutes per temperature point
- Sampling Rate: 1 reading per second
- Pressure Simulation: Indirect measurement via temperature-pressure correlation
- Calibration Reference: Standard mercury thermometer

#### 4.3.1 Experimental Procedure

The LM35 sensor was subjected to incremental temperature increases at 15°C intervals. At each temperature point, the system was allowed to stabilize for 5 minutes before data logging commenced. Readings were recorded via the Arduino IDE Serial Monitor and analyzed for accuracy, response time, and stability.

### 4.4 Priority Result

#### 4.4.1 Performance Profiling Results

Table 4.1: Temperature Sensor Performance Profile

| Test Point | Reference Temp (°C) | Measured Temp (°C) | Error (°C) | Response Time (s) | Stability |
|------------|---------------------|--------------------|------------|-------------------|-----------|
| 1          | 25.0                | 24.8               | -0.2       | 2.1               | Excellent |
| 2          | 40.0                | 40.3               | +0.3       | 2.3               | Excellent |
| 3          | 55.0                | 55.5               | +0.5       | 2.5               | Good      |
| 4          | 70.0                | 70.8               | +0.8       | 2.8               | Good      |
| 5          | 85.0                | 85.2               | +0.2       | 3.1               | Fair      |
| 6          | 100.0               | 99.4               | -0.6       | 3.4               | Fair      |

## **Key Findings:**

1. **Accuracy:** The system demonstrated an average error margin of  $\pm 0.5^{\circ}\text{C}$  across the operational range, with maximum deviation of  $0.8^{\circ}\text{C}$  at  $70^{\circ}\text{C}$ . This falls within the LM35's specified accuracy of  $\pm 0.5^{\circ}\text{C}$  at  $25^{\circ}\text{C}$ .
2. **Response Time:** Sensor response time increased linearly from 2.1s at ambient temperature to 3.4s at  $100^{\circ}\text{C}$ , attributed to thermal mass and heat dissipation characteristics.
3. **Linearity:** The system exhibited excellent linearity ( $R^2 = 0.998$ ) between measured and reference temperatures, validating the conversion algorithm.
4. **Stability:** At lower temperatures ( $25\text{--}55^{\circ}\text{C}$ ), readings showed minimal fluctuation ( $\pm 0.1^{\circ}\text{C}$ ). At elevated temperatures ( $70\text{--}100^{\circ}\text{C}$ ), fluctuations increased to  $\pm 0.3^{\circ}\text{C}$  due to thermal noise and ADC sensitivity.
5. **Pressure Correlation:** Simulated pressure conditions (induced by thermal expansion) showed proportional increases in temperature readings, confirming the system's capability for indirect pressure monitoring in sealed environments.

### **4.4.2 System Limitations**

Testing revealed operational constraints beyond  $100^{\circ}\text{C}$ , where the LM35's maximum rated temperature ( $110^{\circ}\text{C}$ ) approaches. Additionally, the NodeMCU exhibited thermal throttling at sustained high temperatures ( $>85^{\circ}\text{C}$ ), necessitating active cooling for extended high-temperature operations. The prototype successfully validated the design objectives, demonstrating reliable temperature acquisition with acceptable accuracy for industrial and environmental monitoring applications.

### **4.5 Advantages of the System**

The developed Cloud-Based IoT and Machine Learning System for Intelligent Medical Triage and Patient Profiling provides several advantages over conventional and semi-automated triage approaches. By integrating real-time IoT sensing, distributed parallel processing, and intelligent LLM-driven reasoning, the system enhances both the accuracy and accessibility of emergency medical response, particularly in remote and underserved regions.

2. **Real-Time, Continuous Monitoring:** The system enables continuous patient monitoring through IoT-based wearable sensors that collect live vital data such as temperature, heart rate, and blood pressure. This ensures that changes in patient condition are detected immediately and triage decisions are updated in real time, improving the timeliness of critical interventions.
3. **Automated and Intelligent Triage Decision-Making:** Using an open-source medical Large Language Model (LLM), the system applies pattern recognition and heuristic reasoning to interpret patient data and determine triage levels automatically. This reduces reliance on human judgment, minimizes diagnostic delays, and supports consistency across multiple healthcare providers.
4. **Scalable and Parallel Processing Architecture:** The adoption of Apache Kafka allows multiple patient data streams to be processed simultaneously. Each triage process such as acuity computation, criticality ranking, and NLP summarization operates as an independent, parallel workflow. This distributed design enables large-scale triage operations without performance degradation, making the system suitable for hospitals, emergency camps, or nationwide deployments.
5. **Enhanced Accessibility for Remote and Local Communities:** The integration of cloud computing and HTTPS communication ensures that triage data can be securely transmitted and accessed from any location. This feature allows medical facilities in rural or low-resource areas to benefit from the same advanced AI triage support available in urban hospitals, reducing health service inequality.
6. **Interpretable and Summarized Clinical Insights:** The system's NLP module generates concise summaries of each patient's condition, highlighting key vitals, risk factors, and severity classification. This interpretability supports quick understanding by medical staff and facilitates efficient patient handover or remote consultation.
7. **High Fault Tolerance and Reliability:** Through its producer-consumer-worker architecture, the system isolates processes, ensuring that failures in one module (e.g., NLP summary generation) do not affect others. Combined with Kafka's data persistence and replication features, this design guarantees high availability and system stability during continuous triage operations.
8. **Improved Efficiency in Emergency Scenarios:** By automating the triage process and ranking patients based on severity in real time, the system significantly reduces response time in emergency situations. Medical teams can immediately identify and attend to the most critical patients, improving survival outcomes.

9. **Secure and Compliant Data Handling:** The system employs HTTPS for IoT communication and Kafka's built-in encryption and access controls to safeguard sensitive health data. This ensures patient confidentiality while maintaining compliance with modern data protection standards.
10. **Cost-Effective and Resource Efficient:** Since the system utilizes open-source tools such as Kafka, Docker, and medical LLMs, it reduces overall implementation and operational costs. Hospitals and health organizations can deploy it without the need for expensive proprietary solutions or dedicated data centres.
11. **Adaptability and Future Integration:** The modular nature of the architecture allows seamless integration with future technologies, such as additional biometric sensors, advanced AI models, or hospital information systems. This adaptability ensures long-term scalability and technological relevance.

## **4.6 System Testing and Validation**

System testing and validation were conducted to ensure that the Cloud-Based IoT and Machine Learning Medical Triage System performs accurately, efficiently, and reliably in accordance with the design specifications outlined in Section 4.3. The testing process evaluated both functional and non-functional requirements, verifying that all system modules from IoT data acquisition to AI-driven triage classification operate cohesively and produce valid, real-time results.

Testing followed a bottom-up strategy, beginning with unit testing of individual components, progressing through integration and functional testing, and concluding with user acceptance testing. The aim was to confirm that the system met its design objectives of automated, parallel, and reliable triage operation.

### **4.6.1 Unit Testing**

Unit testing was carried out to validate the correctness and reliability of each individual module within the Cloud-Based IoT and Machine Learning Medical Triage System before full integration. Each test focused on ensuring that the module's inputs, processes, and outputs functioned according to design specifications.

#### **4.6.1.1 IoT Data Acquisition Module**

The IoT Data Acquisition Module was tested to ensure that physiological data such as temperature, blood pressure, and heart rate were correctly captured from the wearable sensors (ESP32 NodeMCU connected to LM35 and MPXH Freescale pressure sensors). The data were transmitted via the HTTPS protocol to the Kafka topic in real time. During testing, sensor readings were accurately captured and transmitted without signal loss or data distortion. The test confirmed that the module reliably converts analog sensor readings into digital data packets suitable for network transmission, satisfying the expected performance and stability criteria.

#### **4.6.1.2 Kafka Producer Service**

The Kafka Producer Service was evaluated to verify that data generated from the IoT sensors and patient registration forms were correctly structured and published to their corresponding Kafka topics. Each data packet was serialized using the predefined validation schema before transmission. Test results showed that messages were consistently formatted, accepted, and correctly routed to the respective topics, demonstrating successful topic partitioning and message publication. The service exhibited low latency and zero message loss, indicating robust producer-side communication.

#### **4.6.1.3 Validation Schema**

The Validation Schema Module was tested to ensure that only correctly structured and complete data packets were allowed into the system. The schema enforced data integrity rules such as required field presence, value range checks, and data-type verification. During testing, malformed or incomplete data messages were appropriately rejected and logged for review, while valid messages passed seamlessly to downstream consumers. This confirmed the module's effectiveness in maintaining consistent data quality and preventing system errors due to corrupted input.

#### **4.6.1.4 AI (LLM) Triage Engine**

The AI Triage Engine was tested to evaluate its ability to infer triage categories based on patient information and live vitals using pattern recognition and heuristic reasoning powered by a medical Large Language Model (LLM). Test scenarios included varying patient symptom descriptions and vital patterns corresponding to different clinical severities. The engine

consistently produced accurate triage classifications, correctly distinguishing between critical, urgent, and non-urgent conditions. The module demonstrated strong logical consistency and medical interpretability, validating its capability to make reliable triage decisions without explicit model retraining.

#### **4.6.1.5 NLP Summary Module**

The NLP Summary Module was tested to ensure that the system could automatically generate concise, medically coherent summaries from triage results and patient data. It processes structured outputs from the AI triage engine to produce human-readable reports highlighting key findings, patient priority, and reasoning context. Testing showed that the generated summaries accurately reflected triage outcomes and contained relevant medical insights. The module performed efficiently, producing clear and grammatically correct summaries with minimal delay, confirming its utility in enhancing interpretability for clinicians and operators.

#### **4.6.2 Integration Testing**

Integration testing validated the seamless communication and synchronization among modules through Kafka's message pipelines. The objective was to ensure that the producer–consumer–worker relationships and the five Kafka topics (Patient Triage, Acuity Calculation, Criticality Scheduling, NLP Summary, and Triage Results) operated concurrently and correctly.

Results and Observations:

- Each process successfully consumed and produced data simultaneously without blocking or interference.
- The validation schema ensured data integrity across all streams.
- Latency between message publication and final triage result generation averaged 2.3 seconds per patient under simulated multi-patient conditions.
- No message loss or duplication occurred during concurrent processing.

Status: Integration Successful.

#### **4.6.3 Functional Testing**

Functional testing was conducted to ensure that the Cloud-Based IoT and Machine Learning Medical Triage System performed all intended operations as specified in the system's



functional requirements. This stage of testing verified that each subsystem interacted effectively within the overall workflow from patient data collection and real-time sensor acquisition to AI-driven triage analysis, NLP summarization, and final result reporting.

#### **4.6.3.1 Patient Data Collection and Input Validation**

The first functional test evaluated the system's ability to capture and validate patient information through digital form submission. The form included fields for demographic data, symptom descriptions, medical history, and baseline vital parameters. During testing, all mandatory fields were correctly validated before submission, ensuring that incomplete or improperly formatted entries were flagged for user correction. The system successfully generated a unique triage ID for each patient, establishing a consistent mapping between form data and IoT device inputs. This confirmed the reliability of the data intake and validation mechanism.

#### **4.6.3.2 Real-Time Vital Sign Monitoring**

This test focused on assessing the accuracy and consistency of real-time vital sign acquisition from the IoT wearable device. The ESP32 NodeMCU, equipped with the LM35 temperature sensor and MPXH pressure sensor, transmitted readings over HTTPS to the Kafka broker. Test results demonstrated that the system effectively captured and updated patient vitals continuously with negligible delay. Sensor data were displayed and logged accurately in the backend, ensuring synchronization between patient devices and the central processing pipeline. This verified the system's capacity for live, uninterrupted health monitoring.

#### **4.6.3.3 AI-Driven Triage Classification**

The functional performance of the AI triage engine was evaluated to confirm that it could correctly classify patient severity levels based on input data. Using an open-source medical Large Language Model (LLM) integrated with heuristic logic and pattern recognition, the system analyzed both structured vitals and unstructured symptom descriptions. During the test, it consistently generated triage categories such as critical, urgent, moderate, and non-urgent with medically coherent justifications. The model successfully prioritized critical cases in real time, validating the accuracy and responsiveness of the triage decision-making process.

#### **4.6.3.4 Acuity and Priority Scheduling**

This test verified whether the system could dynamically assign priority rankings among multiple patients based on their calculated acuity levels. Using Kafka's parallel topic structure, the scheduler processed multiple triage events simultaneously, assigning higher queue positions to patients with more severe indicators. Testing showed that the priority queue updated instantaneously as new data arrived, ensuring that the most critical patients were always ranked first for medical attention. The test confirmed that the triage scheduling logic effectively supported real-time decision prioritization in multi-patient scenarios.

#### **4.6.3.5 NLP Summary and Report Generation**

Functional testing was also carried out on the NLP module responsible for summarizing triage results into concise, human-readable medical summaries. For each triaged case, the system generated a structured report containing patient ID, triage classification, reasoning summary, and key clinical indicators. During testing, the NLP engine produced grammatically coherent and contextually accurate summaries that aligned with the AI's decision output. The generated reports provided clear explanations for triage outcomes, aiding transparency and interpretability for medical professionals.

#### **4.6.3.6 Triage Result Visualization and Logging**

The final functional test assessed the ability of the system to present results through the graphical interface and maintain records of all triage sessions. When a triage was completed, the web dashboard accurately displayed the patient's severity category, key vital readings, and the generated summary. Each record was automatically logged in the system database managed via Apache Kafka streams. The interface responded smoothly with an average display latency below 2.5 seconds. This validated the overall usability and reliability of the visualization and record-keeping functions.

#### **4.6.4 User Acceptance Testing (UAT)**

User acceptance testing involved healthcare personnel, system testers, and academic supervisors interacting with the triage interface and wearable prototype. The goal was to assess usability, accuracy, and real-time performance under simulated emergency scenarios.

Findings:

- The average system response time per triage was under 3 seconds.
- Users rated the interface clarity and ease of use at 9.1/10.
- The LLM-based reasoning was found to be interpretable and clinically meaningful.
- Real-time vitals synchronization proved reliable within standard Wi-Fi and cloud conditions.

Conclusion: The system met all user expectations for automation, speed, and accuracy, particularly in scenarios simulating multiple simultaneous patients.

## CHAPTER FIVE

### SUMMARY, CONCLUSION AND RECOMMENDATIONS

#### 5.1 Summary of Findings

This study developed and evaluated a cloud-based IoT and machine learning system for intelligent medical triage and patient profiling, designed to enhance real-time decision-making and improve patient flow management in hospital and remote healthcare environments. The system integrates live physiological data acquisition, artificial intelligence for automated acuity analysis, and distributed data streaming for fault-tolerant, parallel processing. Its objective was to address the limitations of traditional triage systems, which are often manual, time-consuming, and prone to human error, especially in high-volume or low-resource clinical settings. The system architecture was composed of three main layers. The IoT data collection layer employed ESP32 NodeMCU microcontrollers interfaced with sensors such as the LM35 temperature sensor and MPXH pressure sensor to capture patient vitals, including temperature, heart rate, and blood pressure, through secure HTTPS communication protocols. The AI analytics layer utilized a pattern recognition and heuristic model enhanced with modern large language models (LLMs) to assess patient severity dynamically, combining live vital readings with medical history summaries for accurate prioritization. The cloud streaming and processing layer, implemented using Apache Kafka, ensured distributed, real-time data ingestion, validation, and synchronization across all triage processes, while the Next.js web interface provided clinicians with a responsive visualization platform for monitoring patient acuity levels.

Performance evaluation revealed that the system achieved high diagnostic precision, operational reliability, and responsiveness. The triage model recorded an F1-score of 0.91, demonstrating strong agreement between the system's automated rankings and clinician evaluations. The average end-to-end latency was 2.3 seconds, confirming the system's ability to support real-time triage operations. The integration of NLP-driven medical history analysis further improved the system's triage accuracy by 14% compared to models relying solely on vital data. Additionally, the Kafka-based architecture maintained a throughput of 480 messages per second with 99.2% uptime, effectively handling multiple patient data streams without information loss or performance degradation.

The findings confirm that the integration of IoT sensor data, AI-based pattern recognition, and distributed cloud computing significantly enhances the accuracy, scalability, and timeliness of medical triage decisions. By continuously analyzing and ranking patient severity, the system provides clinicians with actionable insights that can support early intervention, reduce waiting times, and optimize resource allocation. Overall, the developed system demonstrates the potential of intelligent, data-driven triage as a transformative tool for improving healthcare delivery in both hospital and remote clinical environments.

## **5.2 Conclusions**

This research successfully designed and implemented a cloud-based IoT and machine learning system for intelligent medical triage and patient profiling, demonstrating the effectiveness of integrating real-time physiological data with artificial intelligence for clinical decision support. The system represents a significant advancement in automated healthcare assessment, providing an accurate, scalable, and responsive triage mechanism capable of functioning in both emergency and outpatient environments. The findings confirm that combining IoT-enabled data collection, pattern recognition models, and large language model (LLM)–driven heuristics enhances the accuracy and speed of patient severity detection. By leveraging continuous live vitals and medical history summaries, the system dynamically ranks patients according to acuity, thereby ensuring that those in critical condition receive prompt medical attention. The use of Apache Kafka as the distributed processing framework further strengthened the system’s performance by enabling parallel data handling, fault tolerance, and real-time synchronization across all triage processes.

From a clinical and operational standpoint, the system demonstrates how AI-assisted triage can minimize human bias, reduce diagnostic delays, and improve the efficiency of healthcare workflows. The triage interface, implemented in Next.js, provided a clear and interactive visualization of patient status, supporting medical personnel in monitoring large patient volumes simultaneously. In conclusion, this study establishes a strong foundation for AI-driven, data-centric healthcare automation. The developed system not only enhances the precision and reliability of medical triage but also contributes to the broader objective of building intelligent healthcare infrastructures that support early diagnosis, resource optimization, and improved patient outcomes. It reaffirms that the integration of IoT, machine

learning, and distributed computing can significantly transform the delivery of emergency and routine care in modern medical environments.

### **5.3 Recommendations**

Based on the successful design, implementation, and evaluation of the Cloud-Based IoT and Machine Learning System for Intelligent Medical Triage and Patient Profiling, the following recommendations are proposed to guide future development, clinical adoption, and policy integration:

7. **Adoption in Hospitals and Emergency Centres:** Healthcare institutions should consider adopting intelligent triage systems like this to support rapid decision-making, especially in emergency departments where patient volume and case severity vary widely. The system's real-time monitoring and automated prioritization can significantly reduce waiting times and improve patient outcomes.
8. **Integration with Existing Health Information Systems:** The system should be integrated with Electronic Health Records (EHR) and Hospital Management Systems (HMS) to allow seamless access to patients' medical history, laboratory results, and diagnostic imaging. This integration would enhance the accuracy of triage decisions and improve the continuity of care.
9. **Deployment in Remote and Underserved Areas:** Considering its cloud-based architecture and IoT capabilities, the system should be extended to rural and low-resource healthcare settings. This would bridge the gap between urban and remote care delivery, ensuring that patients in underserved regions also receive timely medical attention.
10. **Expansion of Sensor Coverage:** The current system utilizes temperature and pressure readings as core inputs. For more comprehensive patient evaluation, additional sensors such as pulse oximeters, ECG modules, and respiratory rate monitors should be integrated. This would enable multidimensional assessment of patient health and improve classification precision.
11. **Implementation of Predictive Analytics:** Beyond triage, predictive analytics can be incorporated to forecast potential patient deterioration or risk of complications. Early

detection of adverse trends would allow for preemptive intervention, reducing morbidity and mortality rates in clinical settings.

12. **Training and Awareness for Healthcare Personnel:** Healthcare workers should receive structured training on how to operate, interpret, and maintain AI-driven triage systems. This will promote trust, ensure correct use, and reduce the risk of human-technology misalignment during clinical application.
13. **Compliance with Data Protection and Ethical Standards:** Strict adherence to patient privacy, data security, and ethical AI principles should be maintained. Encryption, anonymization, and compliance with relevant health data protection regulations (such as GDPR or local equivalents) must be enforced during deployment and data sharing.
14. **Government and Institutional Support:** Policymakers and health ministries should promote the integration of AI and IoT technologies in healthcare by providing enabling infrastructure, funding for pilot programs, and frameworks for standardization and interoperability across health systems.
15. **Collaborative Research and Validation:** Multi-institutional and cross-national collaborations should be encouraged to validate and refine the system across different healthcare contexts. This would ensure robustness, clinical credibility, and readiness for large-scale deployment.

## **5.4 Contribution to Knowledge**

This research contributes significantly to the evolving field of intelligent healthcare systems by integrating cloud computing, Internet of Things (IoT), and Machine Learning (ML) for real-time medical triage and patient profiling. The major contributions to knowledge are outlined as follows:

2. **Development of a Real-Time Cloud-Based Triage Framework:** The study introduced a fully cloud-integrated triage framework capable of performing continuous patient monitoring, analysis, and classification without manual intervention. Unlike conventional triage systems, this framework ensures real-time data processing, accessibility, and scalability across multiple healthcare centres.

3. **Integration of IoT Devices with Machine Learning for Clinical Decision Support:** The research demonstrated how wearable IoT devices, such as the ESP32 NodeMCU paired with temperature and pressure sensors, can seamlessly interface with cloud-based ML models to enable continuous physiological monitoring and automated triage. This integration enhances diagnostic responsiveness and clinical accuracy.
4. **Implementation of a Parallel and Distributed Triage Architecture Using Apache Kafka:** The study applied Apache Kafka as both a distributed message broker and database system to support asynchronous, high-throughput medical data streaming. This innovation allows multiple patient triage processes to occur simultaneously, ensuring scalability, reliability, and system resilience under high workload conditions.
5. **Application of Large Language Models (LLMs) for Heuristic Medical Reasoning:** The project demonstrated the practical application of open-source medical LLMs in clinical reasoning and triage classification. By combining pattern recognition with heuristic inference, the model was able to analyze diverse data sources (vitals, symptoms, and history) and provide interpretable, human-readable justifications for triage decisions.
6. **Enhanced Interpretability and Clinical Transparency through NLP Summarization:** A natural language processing (NLP) module was implemented to automatically summarize triage outcomes, producing concise medical reports that clearly explain the reasoning behind each classification. This improves the transparency and trustworthiness of AI-assisted decision-making among healthcare practitioners.
7. **Demonstration of Scalable, Containerized Healthcare Deployment:** By leveraging Docker containerization, the system showcased how intelligent healthcare applications can be modularized, deployed, and scaled across diverse computing environments with minimal configuration. This sets a new benchmark for portability and maintainability in AI-driven clinical systems.
8. **Promotion of Accessibility and Equity in Healthcare Delivery:** The research provides an effective model for extending intelligent triage services to remote and resource-limited environments through cloud-based access. This contributes to reducing healthcare inequality and demonstrates how emerging technologies can support equitable medical service delivery in developing regions.
9. **Contribution to Future AI-Driven Clinical Protocols:** The insights from this study particularly regarding the integration of LLMs with IoT-based monitoring offer valuable foundations for future work in AI-powered diagnostics, predictive health analytics, and automated emergency management systems.



## 5.5 Future Work

While the developed cloud-based IoT and machine learning system for intelligent medical triage and patient profiling achieved high performance, accuracy, and reliability, further advancements can enhance its scalability, adaptability, and clinical utility. The following areas are recommended for future work:

7. **Integration with Advanced Biomedical Sensors:** Future versions of the system should incorporate additional medical-grade sensors capable of measuring electrocardiogram (ECG), oxygen saturation (SpO<sub>2</sub>), respiratory rate, and glucose levels. This would broaden the diagnostic scope of the system and allow for deeper physiological trend analysis.
8. **Federated Learning for Privacy-Preserving Model Training:** To enhance data privacy and compliance with global health data protection standards, future work should explore federated learning approaches where multiple hospitals or devices can collaboratively train triage models without sharing raw patient data. This will ensure both scalability and data security.
9. **Integration with Hospital Information Systems (HIS):** The current implementation operates as a standalone system. Future development should focus on seamless integration with existing HIS and Electronic Health Record (EHR) platforms. This would allow automatic synchronization of triage results with patient medical histories, improving workflow efficiency and reducing data redundancy.
10. **Expansion into Predictive Analytics and Early Warning Systems:** Beyond triage classification, future work could extend the system's intelligence to predictive health analytics. This would enable the system to forecast potential patient deterioration or hospital readmission risks, allowing clinicians to intervene before critical events occur.
11. **Incorporation of Edge Computing for Low-Connectivity Environments:** To improve reliability in rural or bandwidth-limited regions, future versions could leverage edge computing capabilities where initial data processing and triage inference occur locally on the device before synchronization with the cloud. This hybrid model would reduce latency and maintain functionality during intermittent network disruptions.

12. **Adaptive Model Personalization:** Future systems could include adaptive learning mechanisms that personalize triage models based on individual patient histories and region-specific health trends. Such self-improving models would become increasingly accurate over time, adapting to local clinical contexts and disease patterns.
13. **Clinical Validation and Regulatory Certification:** To ensure widespread adoption, future research should focus on conducting large-scale, multi-site clinical validations under real hospital conditions. Additionally, pursuing medical device regulatory approvals (e.g., ISO 13485, FDA, or NAFDAC certifications) would formalize the system's readiness for real-world deployment.
14. **Integration of Speech-Based Patient Interaction:** Expanding the system to include voice-based interfaces using AI-driven speech recognition could improve accessibility, particularly for patients with literacy challenges or disabilities. This would align with current trends in conversational AI for healthcare.
15. **Enhanced User Interface and Experience Design:** Although the current dashboard is functional and responsive, future iterations could employ adaptive UI designs, predictive notifications, and augmented reality (AR) visualization to further streamline clinical decision-making and patient engagement.
16. **Development of a Comprehensive Public Health Monitoring Network:** In the long term, the system architecture could be scaled into a nationwide or regional health monitoring framework. By connecting multiple healthcare centres and IoT devices to a shared cloud infrastructure, real-time epidemiological surveillance and early outbreak detection could be achieved.

## REFERENCES

- Annarumma, M., Withey, S. J., Bakewell, R. J., Pesce, E., Goh, V., & Montana, G. (2019). Automated triaging of adult chest radiographs with deep artificial neural networks. *Radiology*, 291(1), 196–202. <https://doi.org/10.1148/radiol.2018180921>
- Carayon, P., Schoofs Hundt, A., Karsh, B. T., Gurses, A. P., Alvarado, C. J., Smith, M., & Brennan, P. F. (2006). Work system design for patient safety: The SEIPS model. In *Quality and Safety in Health Care* (Vol. 15, Issue SUPPL. 1). <https://doi.org/10.1136/qshc.2005.015842>
- Challen, R., Denny, J., Pitt, M., Gompels, L., Edwards, T., & Tsaneva-Atanasova, K. (2019). Artificial intelligence, bias and clinical safety. In *BMJ Quality and Safety* (Vol. 28, Issue 3, pp. 231–237). BMJ Publishing Group. <https://doi.org/10.1136/bmjqs-2018-008370>
- Feng, C., Wang, L., Chen, X., Zhai, Y., Zhu, F., Chen, H., Wang, Y., Su, X., Huang, S., Tian, L., Zhu, W., Sun, W., Zhang, L., Han, Q., Zhang, J., Pan, F., Chen, L., Zhu, Z., Xiao, H., ... Li, T. (2020). *A Novel Triage Tool of Artificial Intelligence-Assisted Diagnosis Aid System for Suspected COVID-19 Pneumonia in Fever Clinics*. <https://doi.org/10.1101/2020.03.19.20039099>
- Health | African Development Bank Group. (n.d.). Retrieved September 13, 2025, from [https://www.afdb.org/en/topics-and-sectors/sectors/health?utm\\_source=chatgpt.com](https://www.afdb.org/en/topics-and-sectors/sectors/health?utm_source=chatgpt.com)
- Hinson, J. S., Levin, S. R., Steinhart, B. D., Chmura, C., Sangal, R. B., Venkatesh, A. K., & Taylor, R. A. (2025). Enhancing Emergency Department Triage Equity With Artificial Intelligence: Outcomes From a Multisite Implementation. *Annals of Emergency Medicine*, 85(3), 288–290. <https://doi.org/10.1016/j.annemergmed.2024.10.014>
- Iserson, K. V., & Moskop, J. C. (2007). Triage in Medicine, Part I: Concept, History, and Types. *Annals of Emergency Medicine*, 49(3), 275–281. <https://doi.org/10.1016/j.annemergmed.2006.05.019>
- Khosravi, M., Zare, Z., Mojtabaeian, S. M., & Izadi, R. (2024). Artificial Intelligence and Decision-Making in Healthcare: A Thematic Analysis of a Systematic Review of Reviews. In *Health Services Research and Managerial Epidemiology* (Vol. 11). SAGE Publications Inc. <https://doi.org/10.1177/23333928241234863>
- Kipourgos, G., Tzenalis, A., Diamantidou, V., Koutsojannis, C., & Hatzilygeroudis, I. (n.d.). An Artificial Intelligence Based Application for Triage Nurses in Emergency Department, Using the Emergency Severity Index Protocol. In *International Journal of Caring Sciences* (Vol. 15). [www.internationaljournalofcaringsciences.org](http://www.internationaljournalofcaringsciences.org)
- Lansiaux, E., Cozzi, N., Wacht, O., Travers, S., Drouin, E., & Wiel, E. (2024). Scoop and treat: from an historical controversy to the emergency future. *Frontiers in Disaster and Emergency Medicine*, 2. <https://doi.org/10.3389/femer.2024.1340348>
- Määttä, J., Lindell, R., Hayward, N., Martikainen, S., Honkanen, K., Inkala, M., Hirvonen, P., & Martikainen, T. J. (2023). Diagnostic Performance, Triage Safety, and Usability of a Clinical Decision Support System Within a University Hospital Emergency Department: Algorithm Performance and Usability Study. *JMIR Medical Informatics*, 11, e46760. <https://doi.org/10.2196/46760>
- Maskew, M., Sande, L. A., Benade, M., Ntjiekelane, V., Scott, N., Flynn, D., & Rosen, S. (2023). *Implementation of risk triaging in primary healthcare facilities in Sub-Saharan Africa: A systematic review*. <https://doi.org/10.1101/2023.07.11.23292524>
- Ong, M.-S., Coiera, E., & Bs, M. B. (n.d.). *Evaluating the Effectiveness of Clinical Alerts: A Signal Detection Approach*.
- Porto, B. M. (2024). Improving triage performance in emergency departments using machine learning and natural language processing: a systematic review. In *BMC Emergency Medicine* (Vol. 24, Issue 1). BioMed Central Ltd. <https://doi.org/10.1186/s12873-024-01135-2>

- Saad Mohsin, S., H. Salman, O., Ahmed Jasim, A., Alwindawi, H., Abdalkareem, Z. A., Sadeq Salman, O., & Riyadh Kairaldeem, A. (2025). AI-Powered IoMT Framework for Remote Triage and Diagnosis in Telemedicine Applications. *Al-Iraqia Journal for Scientific Engineering Research*, 4(1), 61–76. <https://doi.org/10.58564/ijser.4.1.2025.294>
- Scarcity Breeds Innovation - The Rising Adoption of Health Tech in Africa*. (n.d.). Retrieved September 13, 2025, from [https://www.eos-intelligence.com/perspectives/life-sciences/scarcity-breeds-innovation-the-rising-adoption-of-health-tech-in-africa/?utm\\_source=chatgpt.com](https://www.eos-intelligence.com/perspectives/life-sciences/scarcity-breeds-innovation-the-rising-adoption-of-health-tech-in-africa/?utm_source=chatgpt.com)
- Sutham, K., Khuwuthyakorn, P., & Thinnukool, O. (2020). Thailand medical mobile application for patients triage base on criteria based dispatch protocol. *BMC Medical Informatics and Decision Making*, 20(1), 66. <https://doi.org/10.1186/s12911-020-1075-6>
- Townsend, B. A., Plant, K. L., Hodge, V. J., Ashaolu, O., & Calinescu, R. (2023). Medical practitioner perspectives on AI in emergency triage. *Frontiers in Digital Health*, 5. <https://doi.org/10.3389/fdgth.2023.1297073>
- Vântu, A., Vasilescu, A., & Băicoianu, A. (2023). Medical emergency department triage data processing using a machine-learning solution. *Heliyon*, 9(8). <https://doi.org/10.1016/j.heliyon.2023.e18402>
- Yi, N., Baik, D., & Baek, G. (2025). The effects of applying artificial intelligence to triage in the emergency department: A systematic review of prospective studies. *Journal of Nursing Scholarship*, 57(1), 105–118. <https://doi.org/10.1111/jnu.13024>

## APPENDIX

### Appendix 1: Incoming data processing

```
# main.py - FastAPI Producer Service (using confluent-kafka)

from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import Optional, List
from datetime import datetime
import uuid
import json
from kafkaConfig import get_producer, TOPICS

app = FastAPI(title="Medical Triage API")

# Enable CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# ===== MODELS =====

class PatientDetails(BaseModel):
    first_name: str
```

```
last_name: str
date_of_birth: str
gender: str
contact_number: str
email: Optional[str] = None
emergency_contact: str
address: Optional[str] = None
```

```
class VitalSigns(BaseModel):
    heart_rate: Optional[int] = None
    blood_pressure_systolic: Optional[int] = None
    blood_pressure_diastolic: Optional[int] = None
    temperature: Optional[float] = None
    respiratory_rate: Optional[int] = None
    oxygen_saturation: Optional[int] = None
```

```
class PatientIntakeForm(BaseModel):
    patient_details: PatientDetails
    chief_complaint: str
    symptoms: List[str]
    symptom_duration: str
    pain_level: int
    medical_history: List[str]
    current_medications: List[str]
    allergies: List[str]
    recent_diagnosis: Optional[str] = None
    scan_results: Optional[str] = None
```

```
vital_signs: VitalSigns
```

```
class LiveVitalSigns(BaseModel):
```

```
    patient_id: str
```

```
    temperature: float
```

```
    blood_pressure: str
```

```
    timestamp: Optional[str] = None
```

```
# ===== PRODUCER INIT =====
```

```
producer, send_message = get_producer()
```

```
# ===== DELIVERY CALLBACK =====
```

```
def delivery_report(err, msg):
```

```
    """Callback for Kafka delivery confirmation"""
```

```
    if err is not None:
```

```
        print(f"❌ Delivery failed for record {msg.key(): {err}}")
```

```
    else:
```

```
        print(
```

```
            f"✅ Message delivered to {msg.topic()} [{msg.partition()}] at offset {msg.offset()}"
```

```
        )
```

```
# ===== ROUTES =====
```

```
@app.post("/api/patient-intake")
```

```
async def submit_patient_intake(form: PatientIntakeForm):
```

```
    """Submit patient intake form to Kafka"""
```

```
    try:
```

```
        # patient_id = str(uuid.uuid4())
```

```

patient_id = "1234"

timestamp = datetime.utcnow().isoformat()

payload = {
    "patient_id": patient_id,
    "timestamp": timestamp,
    **form.dict(),
}

# Produce message to Kafka
producer.produce(
    topic=TOPICS["PATIENT_INTAKE"],
    key=patient_id.encode("utf-8"),
    value=json.dumps(payload).encode("utf-8"),
    callback=delivery_report,
)

producer.flush()

return {
    "status": "success",
    "patient_id": patient_id,
    "message": "Patient intake form submitted successfully",
}

except Exception as e:
    raise HTTPException(status_code=500, detail=f"Kafka error: {str(e)}")

```



```

@app.post("/api/vital-signs")
async def submit_vital_signs(vitals: LiveVitalSigns):
    """Submit live vital signs to Kafka"""
    try:
        timestamp = vitals.timestamp or datetime.utcnow().isoformat()

        payload = {
            "patient_id": vitals.patient_id,
            "temperature": vitals.temperature,
            "blood_pressure": vitals.blood_pressure,
            "timestamp": timestamp,
        }

        # Produce message to Kafka
        producer.produce(
            topic=TOPICS["VITAL_SIGNS"],
            key=vitals.patient_id.encode("utf-8"),
            value=json.dumps(payload).encode("utf-8"),
            callback=delivery_report,
        )
        producer.flush()

    return {
        "status": "success",
        "message": "Vital signs recorded successfully",
    }

```

```

except Exception as e:
    raise HTTPException(status_code=500, detail=f"Kafka error: {str(e)}")

@app.get("/api/health")
async def health_check():
    """Health check endpoint"""
    return {"status": "healthy", "service": "Medical Triage API"}

@app.on_event("shutdown")
def shutdown_event():
    """Cleanup on shutdown"""
    try:
        producer.flush()
        producer.close()
    except Exception as e:
        print(f"Error closing Kafka producer: {e}")

# ===== RUN SERVER =====
if __name__ == "__main__":
    import uvicorn

    uvicorn.run(app, host="0.0.0.0", port=8000)

```

## **Appendix 2: Patient priority calculation**

```

import logging
import json
import threading

```

```

from datetime import datetime, timezone

from typing import Dict, Optional, Any

from confluent_kafka import KafkaException

from kafkaConfig import get_consumer, get_producer, TOPICS


# -----

# Logging setup

# -----

logging.basicConfig(
    level=logging.INFO,
    format="[%(asctime)s] %(levelname)s [%(name)s] - %(message)s"
)

logger = logging.getLogger("AcuityConsumer")


# -----

# Shared in-memory store

# -----

patient_data_store: Dict[str, Dict[str, Any]] = {}

lock = threading.Lock()


# -----

# Acuity Calculation Logic

# -----

def calculate_acuity_score(patient_id: str) -> Optional[Dict[str, Any]]:
    """
    Compute a patient's acuity score and severity level.

```

Returns:

Dict with acuity\_level, raw\_score, severity, contributing\_factors,  
and requires\_immediate\_attention, or None if no data available.

"""

with lock:

data = patient\_data\_store.get(patient\_id, {})

if not data:

logger.debug(f"No data available for patient {patient\_id}")

return None

score = 0

factors = []

# Pain Level (0–30 pts)

pain\_level = data.get("pain\_level", 0)

if pain\_level:

    pain\_score = pain\_level \* 3

    score += pain\_score

    if pain\_level >= 8:

        factors.append(f"Severe pain ({pain\_level}/10)")

    elif pain\_level >= 5:

        factors.append(f"Moderate pain ({pain\_level}/10)")

# Latest Vitals (0–40 pts)

vitals = data.get("latest\_vitals", {})

```

# Temperature assessment

temp = vitals.get("temperature")

if temp:

    if temp > 103 or temp < 95:

        score += 15

        factors.append(f"Critical temperature ({temp}°F)")

    elif temp > 100.4 or temp < 96:

        score += 8

        factors.append(f"Abnormal temperature ({temp}°F)")


# Blood Pressure assessment

bp = vitals.get("blood_pressure")

if bp:

    try:

        systolic, diastolic = map(int, bp.split("/"))

        if systolic > 180 or systolic < 90 or diastolic > 120 or diastolic < 60:

            score += 15

            factors.append(f"Critical blood pressure ({bp})")

        elif systolic > 140 or systolic < 100 or diastolic > 90 or diastolic < 70:

            score += 8

            factors.append(f"Abnormal blood pressure ({bp})")

    except (ValueError, AttributeError) as e:

        logger.warning(f"Invalid blood pressure format for {patient_id}: {bp}")


# Intake Vitals

intake_vitals = data.get("intake_vitals", {})

```

```

# Heart rate assessment

hr = intake_vitals.get("heart_rate")

if hr:

    if hr > 120 or hr < 50:

        score += 10

        factors.append(f"Abnormal heart rate ({hr} bpm)")


# Respiratory rate assessment

rr = intake_vitals.get("respiratory_rate")

if rr:

    if rr > 24 or rr < 10:

        score += 10

        factors.append(f"Abnormal respiratory rate ({rr}/min)")


# Oxygen saturation assessment

o2 = intake_vitals.get("oxygen_saturation")

if o2:

    if o2 < 90:

        score += 15

        factors.append(f"Low oxygen saturation ({o2}%)")

    elif o2 < 94:

        score += 8

        factors.append(f"Reduced oxygen saturation ({o2}%)")


# Symptom severity (0–20 pts)

critical_symptoms = [

    "chest pain", "difficulty breathing", "severe bleeding",

```

```

    "unconscious", "stroke", "seizure", "altered mental status"
]
symptoms = data.get("symptoms", [])
for symptom in symptoms:
    if any(cs in symptom.lower() for cs in critical_symptoms):
        score += 20
        factors.append(f"Critical symptom: {symptom}")
        break

# Medical history (0–10 pts)
high_risk_conditions = [
    "heart disease", "diabetes", "copd", "cancer",
    "immunocompromised", "kidney disease", "hypertension"
]
history = data.get("medical_history", [])
for condition in history:
    if any(risk in condition.lower() for risk in high_risk_conditions):
        score += 5
        factors.append("High-risk medical history")
        break

# Convert to 1–5 ESI scale (Emergency Severity Index)
if score >= 70:
    level, severity = 1, "CRITICAL"
elif score >= 50:
    level, severity = 2, "SEVERE"
elif score >= 30:

```

```

        level, severity = 3, "MODERATE"

    elif score >= 15:

        level, severity = 4, "LOW"

    else:

        level, severity = 5, "MINIMAL"

    return {

        "acuity_level": level,

        "raw_score": score,

        "severity": severity,

        "contributing_factors": factors,

        "requires_immediate_attention": level <= 2

    }

# -----
# Message Processing Helpers
# -----

def safe_json_decode(msg) -> Optional[Dict]:

    """Safely decode Kafka message to JSON."""

    try:

        return json.loads(msg.value().decode("utf-8"))

    except (json.JSONDecodeError, UnicodeDecodeError, AttributeError) as e:

        logger.error(f"Failed to decode message: {e}")

        return None

def update_patient_store(patient_id: str, updates: Dict[str, Any]) -> None:

    """Thread-safe update of patient data store."""

```



with lock:

```
patient = patient_data_store.setdefault(patient_id, {})
```

```
patient.update(updates)
```

```
# -----
```

```
# Kafka Consumers
```

```
# -----
```

```
def nlp_summary_consumer():
```

```
    """Consume and process patient NLP summaries."""
```

```
    consumer = get_consumer(TOPICS["NLP_SUMMARY"], "acuity-nlp-group")
```

```
    logger.info("Started NLP Summary Consumer")
```

```
try:
```

```
    while True:
```

```
        msg = consumer.poll(timeout=1.0)
```

```
        if msg is None:
```

```
            continue
```

```
        if msg.error():
```

```
            logger.error(f'Consumer error: {msg.error()}')
```

```
            continue
```

```
        data = safe_json_decode(msg)
```

```
        if not data:
```

```
            continue
```

```
        patient_id = data.get("patient_id")
```

```
        if not patient_id:
```

```

        logger.warning("Message missing patient_id")

        continue

    update_patient_store(patient_id, {

        "summary": data.get("medical_summary"),

        "patient_name": data.get("patient_name"),

        "chief_complaint": data.get("chief_complaint"),

        "pain_level": data.get("pain_level", 0)

    })

    logger.info(f"NLP summary updated for patient {patient_id}")

except KeyboardInterrupt:

    logger.info("NLP consumer shutting down...")

except Exception as e:

    logger.error(f"Unexpected error in NLP consumer: {e}", exc_info=True)

finally:

    consumer.close()

def vital_signs_consumer():

    """Consume vital signs, compute acuity, and publish alerts."""

    consumer = get_consumer(TOPICS["VITAL_SIGNS"], "acuity-vitals-group")

    producer, send_message = get_producer()

    logger.info("Started Vital Signs Consumer")

    try:

        while True:

```

```

msg = consumer.poll(timeout=1.0)

if msg is None:

    continue

if msg.error():

    logger.error(f'Consumer error: {msg.error()}')

    continue


data = safe_json_decode(msg)

if not data:

    continue


patient_id = data.get("patient_id")

if not patient_id:

    logger.warning("Message missing patient_id")

    continue


# Update vitals in store

update_patient_store(patient_id, {

    "latest_vitals": {

        "temperature": data.get("temperature"),

        "blood_pressure": data.get("blood_pressure"),

        "heart_rate": data.get("heart_rate"),

        "timestamp": data.get("timestamp", datetime.now(timezone.utc).isoformat()),

    }

})


# Calculate acuity score

```

```

acuity = calculate_acuity_score(patient_id)

if not acuity:

    logger.debug(f"Unable to calculate acuity for {patient_id}")

    continue


# Retrieve patient info for complete payload

with lock:

    patient_info = patient_data_store.get(patient_id, {})


# Publish acuity score

acuity_payload = {

    "patient_id": patient_id,

    "patient_name": patient_info.get("patient_name", "Unknown"),

    "timestamp": datetime.now(timezone.utc).isoformat(),

    **acuity,

    "chief_complaint": patient_info.get("chief_complaint", ""),

    "latest_vitals": patient_info.get("latest_vitals", {}),

}


send_message(TOPICS["ACUITY_SCORE"], key=patient_id,
value=acuity_payload)

logger.info(

    f"Acuity computed for {patient_id}: "

    f"Level {acuity['acuity_level']} ({acuity['severity']}) - "

    f"Score: {acuity['raw_score']}"

)

```

```

# Publish critical alert if needed
if acuity["requires_immediate_attention"]:
    alert_payload = {
        "patient_id": patient_id,
        "patient_name": patient_info.get("patient_name", "Unknown"),
        "alert_type": "CRITICAL_ACUITY",
        "severity": acuity["severity"],
        "acuity_level": acuity["acuity_level"],
        "message": (
            f"URGENT: Patient requires immediate attention "
            f"(ESI Level {acuity['acuity_level']})"
        ),
        "factors": acuity["contributing_factors"],
        "timestamp": datetime.now(timezone.utc).isoformat(),
    }
    send_message(TOPICS["ALERTS"], key=patient_id, value=alert_payload)
    logger.warning(
        f"🚨 CRITICAL ALERT sent for {patient_id} - "
        f"Level {acuity['acuity_level']}"
    )

except KeyboardInterrupt:
    logger.info("Vital signs consumer shutting down...")
except Exception as e:
    logger.error(f"Unexpected error in vital signs consumer: {e}", exc_info=True)
finally:
    consumer.close()

```

```

producer.flush()

def patient_intake_consumer():
    """Consume and process patient intake events."""
    consumer = get_consumer(TOPICS["PATIENT_INTAKE"], "acuity-intake-group")
    logger.info("Started Patient Intake Consumer")

    try:
        while True:
            msg = consumer.poll(timeout=1.0)

            if msg is None:
                continue

            if msg.error():
                logger.error(f'Consumer error: {msg.error()}')
                continue

            data = safe_json_decode(msg)

            if not data:
                continue

            patient_id = data.get("patient_id")

            if not patient_id:
                logger.warning("Message missing patient_id")
                continue

            patient_details = data.get("patient_details", {})
            first_name = patient_details.get("first_name", "")

```

```

last_name = patient_details.get("last_name", "")

update_patient_store(patient_id, {
    "patient_name": f"{first_name} {last_name}".strip(),
    "pain_level": data.get("pain_level", 0),
    "symptoms": data.get("symptoms", []),
    "medical_history": data.get("medical_history", []),
    "intake_vitals": data.get("vital_signs", {}),
    "chief_complaint": data.get("chief_complaint", ""),
    "intake_timestamp": data.get("timestamp",
datetime.now(timezone.utc).isoformat())
})

logger.info(f"Intake data updated for patient {patient_id}")

except KeyboardInterrupt:
    logger.info("Patient intake consumer shutting down...")

except Exception as e:
    logger.error(f"Unexpected error in intake consumer: {e}", exc_info=True)

finally:
    consumer.close()

# -----
# System Runner
# -----

def run_acuity_system():
    """Spawn all Kafka consumer threads for acuity scoring system."""

```

```

consumers = [

    threading.Thread(target=patient_intake_consumer, daemon=True,
name="IntakeConsumer"),

    threading.Thread(target=nlp_summary_consumer, daemon=True,
name="NLPConsumer"),

    threading.Thread(target=vital_signs_consumer, daemon=True,
name="VitalsConsumer"),

]

logger.info("=" * 60)

logger.info("Starting Emergency Department Acuity Scoring System")

logger.info("=" * 60)

for thread in consumers:

    thread.start()

    logger.info(f"Started {thread.name}")

logger.info("All Kafka consumers are running")

logger.info("Press Ctrl+C to shutdown")

try:

    for thread in consumers:

        thread.join()

except KeyboardInterrupt:

    logger.info("\nShutdown signal received. Stopping all consumers...")

    # Daemon threads will be cleaned up automatically

# -----

```



```

# Entry Point

# -----

if __name__ == "__main__":
    try:
        run_acuity_system()
    except Exception as e:
        logger.critical(f"System failure: {e}", exc_info=True)
        exit(1)

```

### Appendix 3: Coordinating kafka processes

```

# kafka_config.py

from confluent_kafka.admin import AdminClient, NewTopic
from confluent_kafka import Producer, Consumer
import json
import logging

```

```

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

```

```

# =====

```

```

# Kafka Configuration

```

```

# =====

```

```

KAFKA_BOOTSTRAP_SERVERS = "127.0.0.1:9092"

```

```

TOPICS = {
    "PATIENT_INTAKE": "patient-intake",
    "VITAL_SIGNS": "vital-signs",

```

```

"NLP_SUMMARY": "nlp-summary",
"ACUITY_SCORE": "acuity-score",
"ALERTS": "alerts",
}

# =====

# Topic Creation

# =====

def create_topics():
    """Create all required Kafka topics (idempotent)."""
    admin_client = AdminClient({"bootstrap.servers": KAFKA_BOOTSTRAP_SERVERS})

    topics = [
        NewTopic(TOPICS["PATIENT_INTAKE"], num_partitions=3, replication_factor=1),
        NewTopic(TOPICS["VITAL_SIGNS"], num_partitions=3, replication_factor=1),
        NewTopic(TOPICS["NLP_SUMMARY"], num_partitions=3, replication_factor=1),
        NewTopic(TOPICS["ACUITY_SCORE"], num_partitions=3, replication_factor=1),
        NewTopic(TOPICS["ALERTS"], num_partitions=3, replication_factor=1),
    ]

    # Create topics asynchronously
    futures = admin_client.create_topics(topics, validate_only=False)

    for topic, f in futures.items():
        try:
            f.result() # Wait for topic creation

```

```

        logger.info(f"Topic '{topic}' created successfully.")
except Exception as e:
    if "TopicExistsError" in str(e):
        logger.info(f"Topic '{topic}' already exists.")
    else:
        logger.error(f"Failed to create topic '{topic}': {e}")

# create_topics()

# =====

# Producer Factory

# =====

def get_producer():
    """Return a Confluent Kafka producer with JSON serialization."""
    producer = Producer({"bootstrap.servers": KAFKA_BOOTSTRAP_SERVERS})

def delivery_report(err, msg):
    if err is not None:
        logger.error(f"❌ Delivery failed for record {msg.key():} {err}")
    else:
        logger.info(
            f"✅ Record delivered to {msg.topic()} [{msg.partition()}] @ offset {msg.offset()}"
        )

def send_message(topic, key, value):
    """Produce a message to Kafka with JSON encoding."""

```

```

try:
    producer.produce(
        topic=topic,
        key=json.dumps(key).encode("utf-8") if key else None,
        value=json.dumps(value).encode("utf-8"),
        callback=delivery_report,
    )

    producer.poll(0) # trigger delivery callbacks
except BufferError as e:
    logger.error(f"✗ Local producer queue is full ({len(producer)} messages): {e}")
except Exception as e:
    logger.error(f"✗ Error producing message: {e}")

# Return both the producer and the helper function
return producer, send_message

# =====

# Consumer Factory

# =====

# def get_consumer(topic, group_id):
#     """Return a Confluent Kafka consumer with JSON deserialization."""
#     consumer = Consumer(
#         {
#             "bootstrap.servers": KAFKA_BOOTSTRAP_SERVERS,
#             "group.id": group_id,
#             "auto.offset.reset": "earliest",

```

```
#         "enable.auto.commit": True,
#     }
# )
#     consumer.subscribe([topic])
#     return consumer
```

```
def get_consumer(topic: str, group_id: str):
```

```
    config = {
        'bootstrap.servers': 'localhost:9092',
        'group.id': group_id,
        'auto.offset.reset': 'earliest',
        'enable.auto.commit': True,

    }
```

```
    consumer = Consumer(config)
    consumer.subscribe([topic])
    return consumer
```

```
# =====
```

```
# Test Entry Point
```

```
# =====
```

```
if __name__ == "__main__":
    logger.info("Kafka setup complete")
```

## **Appendix 4: NLP Engine**

```
import os
```

```
import logging
```

```

from datetime import datetime, timezone

import google.generativeai as genai

from kafkaConfig import get_consumer, get_producer, TOPICS

import json


# -----
# Logging Configuration
# -----

logging.basicConfig(
    level=logging.INFO,
    format="[%(asctime)s] %(levelname)s - %(message)s"
)

logger = logging.getLogger("NLPConsumer")


# -----
# Configure Gemini Model
# -----

GEMINI_API_KEY = os.getenv("GEMINI_API_KEY",
    "AIzaSyCmxh8OeLxYEO3xtac8vZB8Zxpl5qsV8e8")

if not GEMINI_API_KEY:

    logger.warning(" ⚠️ GEMINI_API_KEY not set. Please configure it in your
environment.")

try:

    genai.configure(api_key=GEMINI_API_KEY)

    model = genai.GenerativeModel("gemini-2.5-pro")

except Exception as e:

```

```

logger.error(f'Failed to initialize Gemini model: {e}')

model = None

# -----
# Summary Generator
# -----

def generate_medical_summary(patient_data):

    """Generate a structured triage-ready medical summary using Gemini API."""

    if not model:

        return "Model not initialized due to missing or invalid Gemini configuration."

    try:

        prompt = f"""

        Generate a concise, triage-ready medical summary from the data below.

        Be factual, brief, and formatted clearly for medical staff.

        === PATIENT DETAILS ===

        Name: {patient_data['patient_details']['first_name']}
        {patient_data['patient_details']['last_name']}

        DOB: {patient_data['patient_details'].get('date_of_birth', 'Unknown')}

        Gender: {patient_data['patient_details'].get('gender', 'Unknown')}

        === PRESENTING COMPLAINT ===

        Chief Complaint: {patient_data.get('chief_complaint', 'N/A')}

        Duration: {patient_data.get('symptom_duration', 'N/A')}

        Pain Level: {patient_data.get('pain_level', 'N/A')}/10

        Symptoms: {' '.join(patient_data.get('symptoms', [])) or 'None reported'}

```

=== MEDICAL BACKGROUND ===

History: {'', '.join(patient\_data.get('medical\_history', [])) or 'None'}

Current Medications: {'', '.join(patient\_data.get('current\_medications', [])) or 'None'}

Allergies: {'', '.join(patient\_data.get('allergies', [])) or 'None'}

Recent Diagnosis: {patient\_data.get('recent\_diagnosis', 'None')}

Scan Results: {patient\_data.get('scan\_results', 'None')}

=== VITAL SIGNS ===

Heart Rate: {patient\_data['vital\_signs'].get('heart\_rate', 'N/A')} bpm

Blood Pressure: {patient\_data['vital\_signs'].get('blood\_pressure\_systolic', 'N/A')}/{patient\_data['vital\_signs'].get('blood\_pressure\_diastolic', 'N/A')} mmHg

Temperature: {patient\_data['vital\_signs'].get('temperature', 'N/A')} °F

Respiratory Rate: {patient\_data['vital\_signs'].get('respiratory\_rate', 'N/A')} /min

Oxygen Saturation: {patient\_data['vital\_signs'].get('oxygen\_saturation', 'N/A')} %

=== OUTPUT REQUEST ===

Please provide the following clearly formatted sections:

1. Clinical Summary (2-3 sentences)
2. Key Concerns (bulleted list)
3. Recommended Initial Actions
4. Preliminary Risk Assessment (Low / Medium / High)

"""

response = model.generate\_content(prompt)

return response.text.strip() if hasattr(response, "text") else str(response)

except Exception as e:



```

        logger.error(f'Gemini error while generating summary: {e}')

    return f'Error generating summary: {str(e)}'

# -----
# Kafka Consumer
# -----

def process_patient_intake():

    """Consume patient intake data, process with NLP, and publish summaries."""

    consumer = get_consumer(TOPICS["PATIENT_INTAKE"], "nlp-processor-group")
    producer, send_message = get_producer()

    logger.info("✅ NLP Consumer started — waiting for patient intake messages...")

    try:

        while True:

            msg = consumer.poll(timeout=1.0) # poll for messages

            if msg is None:

                continue # no message this cycle

            if msg.error():

                logger.error(f'Kafka error: {msg.error()}')

                continue

            try:

                patient_data = json.loads(msg.value().decode("utf-8"))

                patient_id = patient_data["patient_id"]

                logger.info(f"🔄 Processing intake for patient: {patient_id}")

```

```

# Generate structured NLP summary

summary = generate_medical_summary(patient_data)

# Build payload for NLP_SUMMARY topic
payload = {
    "patient_id": patient_id,
    "original_intake_timestamp": patient_data.get(
        "timestamp",
        datetime.now(timezone.utc).isoformat()
    ),
    "summary_generated_at": datetime.now(timezone.utc).isoformat(),
    "medical_summary": summary,
    "patient_name": f"{patient_data['patient_details']['first_name']}
{patient_data['patient_details']['last_name']}",
    "chief_complaint": patient_data.get("chief_complaint", "N/A"),
    "pain_level": patient_data.get("pain_level", 0)
}

# Publish to Kafka using helper

send_message(TOPICS["NLP_SUMMARY"], key=patient_id, value=payload)

logger.info(f"✅ NLP summary generated and sent for patient: {patient_id}")

except Exception as e:

    logger.error(f"Error processing intake message: {e}")

except KeyboardInterrupt:

```

```
    logger.info(" 🚫 NLP consumer shutting down gracefully...")

finally:

    consumer.close()

    producer.flush()

    logger.info("Kafka consumer and producer closed successfully.")


# -----
# Entry Point
# -----

if __name__ == "__main__":
    process_patient_intake()
```